

接触式轮廓仪的自动标注

朱宇豪、喻健凯、黄萱

8/2025

摘要

本文针对工件轮廓线参数确定问题，建立了一套基于数据拟合与几何分析的参数求解方法。

首先，基于附件一 sheet1 的测量数据，通过 MATLAB 绘制散点图并采用交互式方法初步识别轮廓线转折点。考虑到轮廓线由直线段和圆弧构成的特点，采用分段最小二乘拟合法，在初步识别区间内进行精细化拟合，建立轮廓线的精确数学模型，进而通过解析几何方法确定最终转折点坐标及相关参数。

针对 sheet2 中工件在倾斜状态下的 17 组测量数据，我们提出基于基准线校正的数据处理方法：以水平方向为基准，计算各测量数据的倾斜角度并进行水平校正。通过整合校正后的全部数据点，采用最小二乘法进行整体拟合，可以显著提高参数估计的精度。

对于不同倾斜角度下的 10 次重复测量数据，我们提出以最长直线段为基准的标准化处理方法：计算各次测量的相对倾斜角，通过求取平均值消除系统误差，建立标准参考角度体系，从而获得更为可靠的参数估计值。

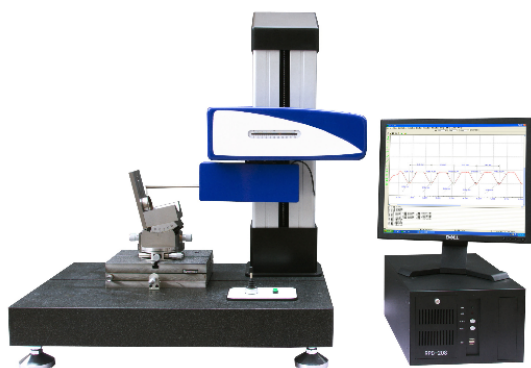
此外，针对 9 组局部测量数据，我们开发了基于特征点匹配的精度提升算法：通过选取关键特征点并进行平移配准，实现多组数据的空间对齐，在此基础上进行融合拟合，进一步提高了参数确定的准确性。该方法为类似工件的几何参数测量提供了可靠的技术方案。

关键词：轮廓线拟合 最小二乘法 几何参数测量 数据校正 特征点

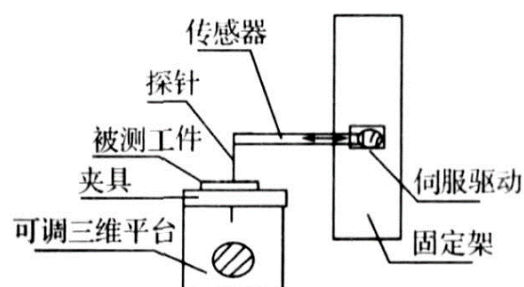
1 问题重述与分析

1.1 问题重述

轮廓仪是一种两坐标测量仪器（见图 1(a)），它由工作平台、夹具、被测工件、探针、传感器和伺服驱动等部件组成（见图 1(b)）。



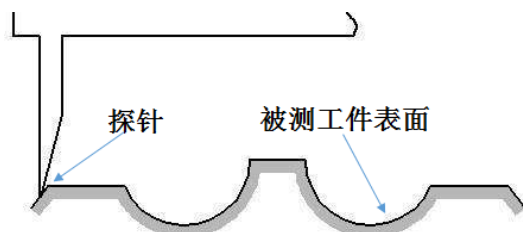
(a) 某种型号的接触式轮廓仪



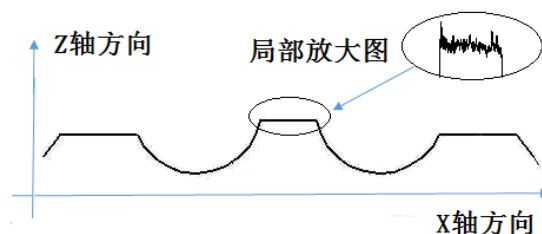
(b) 接触式轮廓仪的示意图

图 1

接触式轮廓仪的工作原理是，探针接触到被测工件表面并匀速滑行，传感器感受到被测表面的几何变化，在 X 和 Z 方向分别采样，并转换成电信号。该电信号经放大等处理，转换成数字信号储存在数据文件中（见图 2）。



(a) 接触式轮廓仪测量示意图



(b) 数据文件中的数字信号

图 2: 接触式轮廓仪的工作原理

在理想状况下，轮廓曲线应该是光滑的，但由于接触式轮廓仪存在探针沾污、探针缺陷、扫描位置不准等问题，检测到的轮廓曲线呈现出粗糙不平的情况（见图 2(b) 中的局部放大图），这给工件形状的准确标注带来影响。为了简化问题，假设被测工件的轮廓线是由直线和圆弧构成的平面曲线（见图 3）。请建立数学模型，并根据附件 1（工件 1 的水平和倾斜测量数据）、附件 2~附件 4（工件 2 的多次测量数据）所提供的轮廓仪测量数据，研究下列问题：

问题 1. 附件 1 中的表 level 是工件 1 在水平状态下的测量数据，其轮廓线如图 4 所示，请标注出轮廓线的各项参数值：槽口宽度（如 x_1, x_3 等）、圆弧半径（如 R_1, R_3 等）、

圆心之间的距离（如 c_1, c_2 等）、圆弧的长度、水平线段的长度（如 x_2, x_4 等）、斜线线段的长度、斜线与水平线之间的夹角（如 $\angle 1, \angle 2$ 等）和人字形线的高度（ z_1 ）。

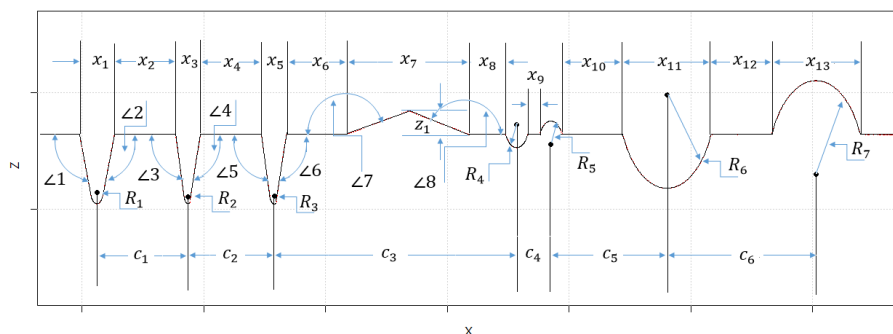


图 3: 工件 1 在水平状态下测量的轮廓线

问题 2. 同一工件在不同次测量时，由于工件放置的角度和位置不同，轮廓线参数的计算值也会存在差异。附件 1 中的表 down 给出了工件 1 在倾斜一个角度和有一些水平位移状态下轮廓线的测量数据。请计算该工件测量时的倾斜角度，并作水平校正。在数据校正后，完成问题 1 的任务，并比较两种测量状态下工件 1 各项参数计算值之间的差异。

问题 3. 在对工件作多次检测时，工件每次放置的角度、测量的起点和终点都会有偏差，这导致了每次测量实际是对整个工件中的某一部分进行检测。附件 2 提供了对工件 2 的 10 次测量数据，请基于这些数据完成：(1) 每次测量时工件 2 的倾斜角度；(2) 标注出工件 2 轮廓线的各项参数值（同问题 1）；(3) 画出工件 2 的完整轮廓线。

问题 4. 为了更准确地标注出工件 2 的各项参数值，附件 3 和附件 4 分别提供了工件 2 关于圆和角的 9 次局部测量数据，请利用这些数据修正问题 3 的结论，并对该工件的完整轮廓线作进一步修正。

1.2 问题分析

问题一: 已经假设工件一轮廓线只由直线段与圆弧构成，于是我们主要采用曲线拟合的方法，利用 matlab 画出附件一水平的散点图，人工初步选取大区间的 17 个分段点，分区间利用最小二乘法以及 LM 算法进行曲线的拟合，得到工件一在水平情况下的分段表达式，为了弥补人工选取分段点的缺陷，我们将 17 个相邻拟合曲线的交点作为正式的间段点（特殊点），再次基础上利用解析几何方法求得工件一水平情况下的参数

问题二: 为了确定工件一的偏转角度与水平位移，我们初步想到了以下两种方法。
方法一: 利用 matlab 画出附件一 sheet2 的散点图，同问题一一样人工选取 17 个初步间段点，以此为区间端点分区间进行曲线的拟合，最终以曲线的 17 个交点作为正式的间段点。对于水平状态以及位移后状态的工件一，我们将两组正式的间段点作为工件一进入

行钢体位移的代表, 以对应特殊点之间的距离为基础, 通过最小二乘法确立最优的偏转角 θ 与水平位移 Δx

方法二: 利用 matlab 画出附件一 sheet2 的散点图, 同方法一一样人工选取 17 个初步分段点, 以此为区间端点进行分段曲线的拟合, 以工件一对应的几组直线作为工件一钢体运动的代表, 分别求出其对应的偏转角, 最终以平均偏转角作为整个工件的偏转角, 将倾斜的工件一水平校正后, 通过分段点的水平位移之差求得整个工件一的水平位移。

问题三: 我们主要利用问题二中方法二的思想对问题三进行处理。对十组数据先画出它们的散点图, 再使用问题二的方法对它们分别进行分段曲线的拟合, 选取它们相对应的直线段计算偏转角与水平位移, 在水平校正以及水平平移后将十组分段函数求和取平均作为对真实的附件一水平状态的数学表示。利用问题一的模型进一步求得其参数。

问题四: 针对多次局部测量数据融合问题, 我们需解决测量误差和坐标系不统一的影响。通过提取特征点进行数据配准, 结合最小二乘平差和加权融合方法, 在保持几何约束的条件下优化参数估计, 从而提高轮廓线关键参数的测量精度。

2 模型假设与符号说明

2.1 模型假设

1. **轮廓分段性假设:** 假设工件轮廓线可以分解为有限个直线段和圆弧段的组合, 且各段之间在转折点处连续。
2. **刚体运动假设:** 假设工件在测量过程中仅发生平移和旋转, 不产生形变, 满足刚体运动特性:

$$\begin{cases} x' = x \cos \theta - z \sin \theta + \Delta x \\ z' = x \sin \theta + z \cos \theta + \Delta z \end{cases}$$

3. **局部线性化假设:** 对于任意足够小的轮廓段 $[s_i, s_{i+1}]$, 存在线性近似:

$$\exists a, b \in \mathbb{R}, \max_{x \in [s_i, s_{i+1}]} |z(x) - (ax + b)| < \varepsilon$$

4. **圆弧光滑性假设:** 圆弧段曲率 k 满足 Lipschitz 连续条件:

$$|k(s_1) - k(s_2)| \leq L|s_1 - s_2|, \forall s_1, s_2 \in \text{圆弧段}$$

5. **特征不变性假设:** 关键特征点 (如曲率极值点) 在位移前后保持拓扑关系不变:

$$\text{Neighbor}(p_i) \leftrightarrow \text{Neighbor}(q_i)$$

6. **小角度近似假设:** 当偏转角 $\theta < 5^\circ$ 时, 允许采用近似计算:

$$\sin \theta \approx \theta, \cos \theta \approx 1 - \frac{\theta^2}{2}$$

2.2 符号说明

表 1: 自定义符号说明表

符号	含义
a_i	转折点编号 i 的横坐标, $i \in [1, 17] \cap \mathbb{Z}$
(x_i, z_i)	第 i 个特征点在工件坐标系中的坐标
θ	工件绕 y 轴的刚性偏转角度 (弧度制)
$(\Delta x, \Delta z)$	工件在 x 和 z 方向的平移量
k_i	离散点 p_i 的近似曲率, 计算式为 $\frac{ (z_{i+1}-z_i)(x_{i-1}-x_i)-(x_{i+1}-x_i)(z_{i-1}-z_i) }{[(x_{i+1}-x_i)^2+(z_{i+1}-z_i)^2]^{3/2}}$
R_j	第 j 段圆弧的拟合半径, $j \in \{1, 3, 5, 7\}$ 对应不同圆弧段
(a_j, b_j)	第 j 段圆弧的圆心坐标, 通过 LM 算法优化得到
$\angle_k$	第 k 段直线的倾斜角, $\angle_k = \pi - \arctan(a_k)$ (a_k 为斜率)
$[a_m, a_n]$	从第 m 个到第 n 个转折点之间的轮廓段
p_i, q_i	分别为基准状态和偏移状态下的匹配特征点对
$\mathcal{E}(\theta, \Delta x, \Delta z)$	刚体变换的最小二乘误差函数: $\sum \ q_i - T(p_i)\ ^2$
$T(\cdot)$	刚体变换算子: $T(p) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} p + \begin{pmatrix} \Delta x \\ \Delta z \end{pmatrix}$
κ_{th}	曲率特征点检测阈值, 取 $\kappa_{th} = 1.5\bar{\kappa}$
$\mathcal{P}_L, \mathcal{P}_C$	分别表示直线段和圆弧段的点集划分

3 模型建立与求解

3.1 工件轮廓线的分段拟合解析模型

3.1.1 初步转折点选取

将附件一经过 matlab 的处理后其散点图如下:

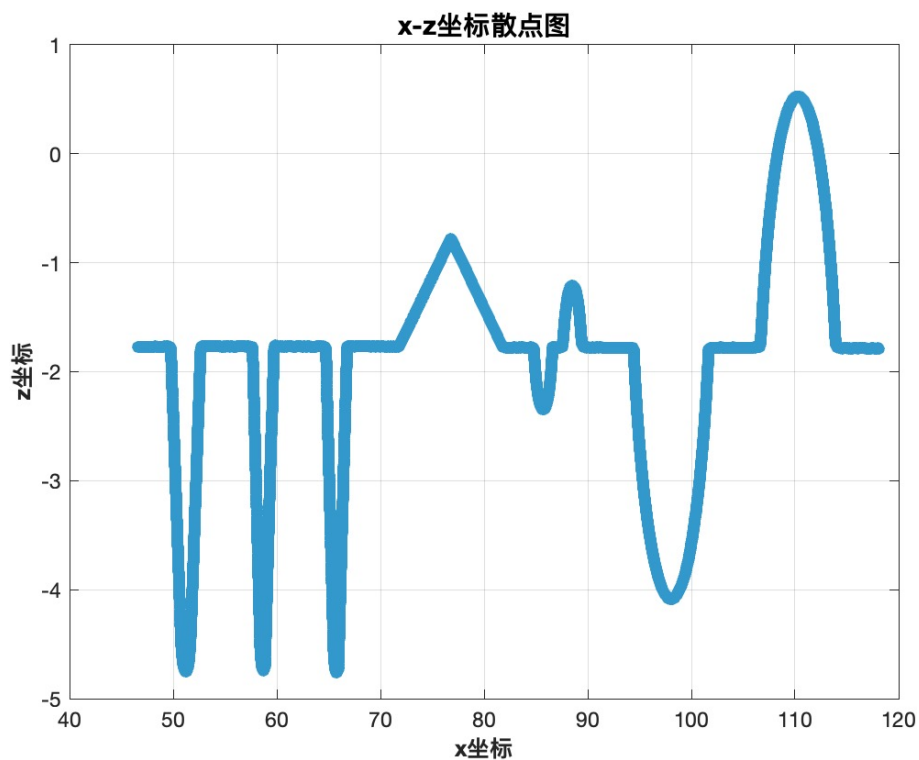


图 4: 水平散点图

检测转折点方式：首先通过绘图手动选取明显的转折点，然后去除掉其 x 轴周围 0.2 的区域，采用区间内部的点进行拟合，最后根据拟合方程计算交点，得到最终转折点。

表 2: 手动粗选的 17 个明细转折点

No.	Position	No.	Position
1	[114.1814, -1.7795]	10	[76.8240, -0.7815]
2	[106.5504, -1.7766]	11	[71.6264, -1.7676]
3	[101.7909, -1.7828]	12	[66.8444, -1.7679]
4	[94.3925, -1.7840]	13	[64.5999, -1.7690]
5	[89.5424, -1.7685]	14	[59.7754, -1.7659]
6	[87.4959, -1.7757]	15	[57.5009, -1.7676]
7	[86.6494, -1.7737]	16	[52.7223, -1.7699]
8	[84.6165, -1.7795]	17	[49.7154, -1.7739]
9	[81.8819, -1.7691]		

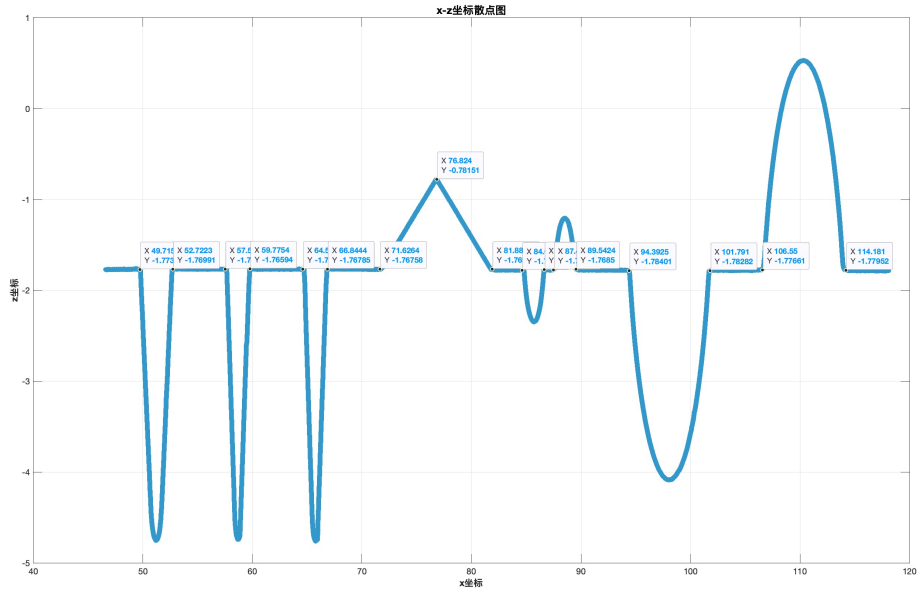


图 5: 初步选取了转折点

直线间交点计算:

$$A_i x + B_i z + C_i = 0,$$

$$\delta = A_1 * B_2 - A_2 * B_1,$$

$$x = (B_1 * C_2 - B_2 * C_1) / \delta,$$

$$z = (A_2 * C_1 - A_1 * C_2) / \delta$$

线圆间交点计算:

$z = mx + c$, 代入圆方程: $(x - h)^2 + (mx + c - k)^2 = r^2$ 求解二次方程, 并设置 δ 的阈值 0.5, 若 δ 的绝对值 < 0.5 , 则视为线圆之间仅有一个切点。最终, 我们选取的转折点如下, 我们依次序从左到右对转折点进行编号 (横坐标 $a_i, i \in [1, 17] \cap \mathbf{Z}$):

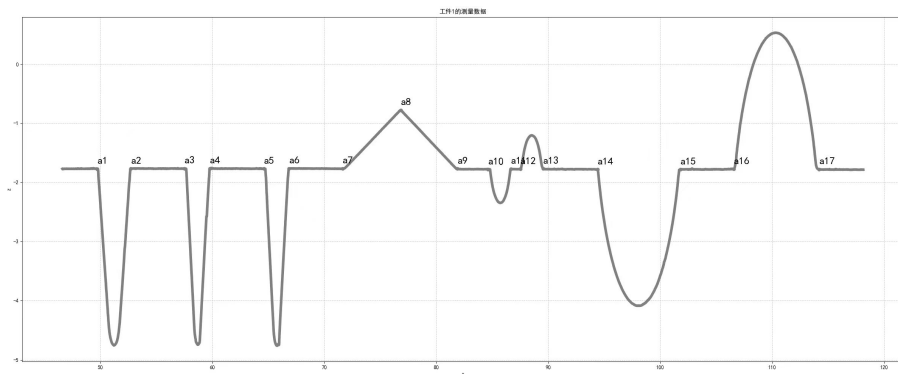


图 6

直线拟合: 根据转折点确定的区间读取数据, 采用 numpy 库的 stats.linregress 函数进行线性拟合。

弧线拟合： 使用数据的质心作为圆心初始估计，计算到质心的平均距离作为半径初始估计，采用 numpy 库的 *least - squares* 中的 'lm' 方法进行迭代。

混合拟合： 第三类拟合中的过渡区间选取办法：在对应区间对 z 轴以 0.1 为间隔统计点的个数，选择点的个数急剧变多的区间拟合圆弧，点的个数变化不大的区间拟合直线，再计算交点得到转折点。

经多次区间判断、实际测试及绘图之后，选取得到最合适的拟合区间如下：

i. $[a_1, a_2]$ ： 选取 z 为 $[-4.3, -4.8]$ 的区域拟合圆弧，取 $[-1.9, -3.8]$ 的区域拟合两端直线。

表 4: Z 值区间分布统计表

区间	点数	区间	点数	区间	点数
$[-4.8, -4.7)$	888	$[-3.5, -3.4)$	140	$[-2.2, -2.1)$	150
$[-4.7, -4.6)$	528	$[-3.4, -3.3)$	144	$[-2.1, -2.0)$	165
$[-4.6, -4.5)$	302	$[-3.3, -3.2)$	130	$[-2.0, -1.9)$	150
$[-4.5, -4.4)$	229	$[-3.2, -3.1)$	160	$[-1.9, -1.8)$	128
$[-4.4, -4.3)$	152	$[-3.1, -3.0)$	153	$[-1.8, -1.7)$	295
$[-4.3, -4.2)$	156	$[-3.0, -2.9)$	146	$[-1.7, -1.6)$	0
$[-4.2, -4.1)$	125	$[-2.9, -2.8)$	142		
$[-4.1, -4.0)$	138	$[-2.8, -2.7)$	145		
$[-4.0, -3.9)$	157	$[-2.7, -2.6)$	144		
$[-3.9, -3.8)$	161	$[-2.6, -2.5)$	141		
$[-3.8, -3.7)$	126	$[-2.5, -2.4)$	143		
$[-3.7, -3.6)$	158	$[-2.4, -2.3)$	140		
$[-3.6, -3.5)$	146	$[-2.3, -2.2)$	133		

ii. $[a_3, a_4]$ ： 选取 z 为 $[-4.3, -4.8]$ 的区域拟合圆弧，取 $[-2.1, -4.2]$ 的区域拟合两端直线。

表 5: Z 值区间分布统计表

z 值区间	点数	z 值区间	点数
$[-4.8, -4.7)$	638	$[-3.6, -3.5)$	119
$[-4.7, -4.6)$	248	$[-3.5, -3.4)$	111
$[-4.6, -4.5)$	163	$[-3.4, -3.3)$	114
$[-4.5, -4.4)$	119	$[-3.3, -3.2)$	108

接下页

表 5 Z 值区间分布统计表 (续)

z 值区间	点数	z 值区间	点数
[-4.4, -4.3)	104	[-3.2, -3.1)	106
[-4.3, -4.2)	105	[-3.1, -3.0)	113
[-4.2, -4.1)	109	[-3.0, -2.9)	117
[-4.1, -4.0)	113	[-2.9, -2.8)	106
[-4.0, -3.9)	108	[-2.8, -2.7)	106
[-3.9, -3.8)	112	[-2.7, -2.6)	110
[-3.8, -3.7)	112	[-2.6, -2.5)	179
[-3.7, -3.6)	98	[-2.5, -2.4)	115

iii. $[a_5, a_6]$: 选取 z 为 [-4.2, -4.8] 的区域拟合圆弧, 取 [-2.1, -4.0] 的区域拟合两端直线。

表 6

z 值区间	点数	z 值区间	点数
[-4.8, -4.7)	741	[-3.6, -3.5)	112
[-4.7, -4.6)	195	[-3.5, -3.4)	111
[-4.6, -4.5)	146	[-3.4, -3.3)	113
[-4.5, -4.4)	115	[-3.3, -3.2)	105
[-4.4, -4.3)	102	[-3.2, -3.1)	113
[-4.3, -4.2)	107	[-3.1, -3.0)	110
[-4.2, -4.1)	110	[-3.0, -2.9)	115
[-4.1, -4.0)	110	[-2.9, -2.8)	110
[-4.0, -3.9)	110	[-2.8, -2.7)	115
[-3.9, -3.8)	107	[-2.7, -2.6)	115
[-3.8, -3.7)	113	[-2.6, -2.5)	104
[-3.7, -3.6)	104	[-2.5, -2.4)	120
[-2.4, -2.3)	87	[-1.9, -1.8)	165
[-2.3, -2.2)	102	[-1.8, -1.7)	367
[-2.2, -2.1)	127	[-1.7, -1.6)	0
[-2.1, -2.0)	93		
[-2.0, -1.9)	146		

在确立了转折点的坐标后, 我们以转折点为界分区间对轮廓线进行曲线拟合。

3.1.2 曲线拟合

这里将不同区间内拟合的曲线 (不考虑水平直线) 类型分为三类:

第一类: 直线段, 有 $[a_7, a_8], [a_8, a_9]$

第二类: 圆周, 有 $[a_{10}, a_{11}], [a_{12}, a_{13}], [a_{14}, a_{15}], [a_{16}, a_{17}]$

第三类: 直线段与圆周, 有 $[a_1, a_2], [a_3, a_4], [a_5, a_6]$

这里就三类情况分别进行处理:

第一类: 水平直线拟合 以区间 $[a_7, a_8]$ 为例。 a_7, a_8 的坐标分别为: (71.6264, -1.7679), (76.8240, -0.7815).

于是区间具体数值为: [71.6264, 76.8240]

采用最小二乘法拟合直线: 设直线方程为 $ax + b$.

目标函数为 $\min f(a, b) = \sum_j (c_j - ab_j - b)^2$

其中 c_j 为区间 $[a_7, a_8]$ 中附件一 sheet1 内点对应的纵坐标, b_j 为其对应的横坐标。

表 7: 拟合直线方程

区间	拟合直线方程
$(-\infty, a_1]$	$z = 0.001316x - 1.833238$
$[a_2, a_3]$	$z = -0.000223x - 1.755124$
$[a_4, a_5]$	$z = -0.000190x - 1.756219$
$[a_6, a_7]$	$z = -0.000877x - 1.706856$
$[a_7, a_8]$	$z = 0.198677x - 16.033946$
$[a_8, a_9]$	$z = -0.200582x + 14.632147$
$[a_9, a_{10}]$	$z = -0.000371x - 1.746974$
$[a_{11}, a_{12}]$	$z = -0.008780x - 1.013106$
$[a_{13}, a_{14}]$	$z = -0.000470x - 1.735078$
$[a_{15}, a_{16}]$	$z = 0.000252x - 1.806660$
$[a_{17}, \infty)$	$z = -0.000948x - 1.674516$

表 8: 拟合统计指标

区间	R^2	p 值	斜率误差	MAE	RMSE	角度 ($^\circ$)	弧度
$(-\infty, a_1]$	0.299	0.000	0.000027	0.001244	0.001583	179.925	3.1403
$[a_2, a_3]$	0.032	0.000	0.000013	0.001257	0.001563	179.987	3.1414
$[a_4, a_5]$	0.021	0.000	0.000014	0.001277	0.001646	179.989	3.1414
$[a_6, a_7]$	0.396	0.000	0.000012	0.001129	0.001371	179.950	3.1407
$[a_7, a_8]$	0.999972	0.000	0.000011	0.001204	0.001457	168.763	2.9455
$[a_8, a_9]$	0.999982	0.000	0.000009	0.000897	0.001130	168.658	2.9436
$[a_9, a_{10}]$	0.038	0.000	0.000027	0.001047	0.001251	179.979	3.1412
$[a_{11}, a_{12}]$	0.582	0.000	0.000249	0.000807	0.000958	179.497	3.1328
$[a_{13}, a_{14}]$	0.118	0.000	0.000014	0.001263	0.001649	179.973	3.1411
$[a_{15}, a_{16}]$	0.017	0.000	0.000020	0.001727	0.002394	179.986	3.1413
$[a_{17}, \infty)$	0.249	0.000	0.000020	0.001291	0.001681	179.946	3.1406

第二类:圆弧拟合 以区间 $[a_{10}, a_{11}]$ 为例, 采用 LM 算法. 其对应的区间为 $[84.6165, 86.6494]$ 设其满足圆方程:

$$(x - a)^2 + (y - b)^2 = r^2$$

其中 (a,b) 是圆心, r 是半径.

定义残差:

$$f_j(a, b, r) = \sqrt{(b_j - a)^2 + (c_j - b)^2} - r$$

目标函数为:

$$\min E(a, b, r) = \min \sum_j f_j^2$$

表 9: 圆弧拟合参数

区间	圆心坐标 (h, k)	半径 r	残差平方和 (RSS)	R^2
$[a_{10}, a_{11}]$	(85.706, -1.350)	0.996	0.016075	0.999716
$[a_{12}, a_{13}]$	(88.517, -2.211)	1.006	0.013345	0.999727
$[a_{14}, a_{15}]$	(98.050, -0.082)	4.003	0.092802	0.999980
$[a_{16}, a_{17}]$	(110.303, -3.478)	4.005	0.250571	0.999957

表 10: 圆弧拟合统计指标

区间	MAE	RMSE	拟合类型
$[a_{10}, a_{11}]$	0.001707	0.002219	arc ₁
$[a_{12}, a_{13}]$	0.001694	0.002013	arc ₂
$[a_{14}, a_{15}]$	0.001908	0.002575	arc ₃
$[a_{16}, a_{17}]$	0.001980	0.004163	arc ₄

第三类：混合区间的拟合 以区间 $[a_1, a_2]$ 为例，其具体数值为 $[49.7154, 52.7223]$ 。

将其分为三部分：第一部分为左侧直线段，第二部分为中间圆弧，第三部分为右侧直线段。

考虑到直线段与圆弧的交点即过渡点难以确认（工件表面粗糙），于是在已有其散点图的情况下将三个部分分别在更小的区间内进行拟合，左侧直线选取的拟合区间为 $[49.8, 50.6]$ ，中间圆弧选取的拟合区间为 $[50.9, 51.4]$ ，右侧直线选取的拟合区间为 $[52.1, 52.7]$ 由此将第三类情形归类为第一类与第二类情形。

表 11: 混合拟合参数（直线 + 圆弧）

区间	拟合类型	方程/圆心	半径/角度	R^2
$[a_1, a_2]$	左侧直线	$z = -2.736517x + 134.435414$	110.074°	0.999818
	右侧直线	$z = 2.618811x - 139.741554$	110.900°	0.994157
	中间圆弧	$(51.218, -4.257), r = 0.495$	-	0.998260
$[a_3, a_4]$	左侧直线	$z = -3.741219x + 213.957262$	104.965°	0.999450
	右侧直线	$z = 3.514363x - 211.547270$	105.884°	0.999947
	中间圆弧	$(58.702, -4.425), r = 0.315$	-	0.931672
$[a_5, a_6]$	左侧直线	$z = -3.642133x + 233.954398$	105.353°	0.997912
	右侧直线	$z = 3.461037x - 232.894114$	106.116°	0.998295
	中间圆弧	$(65.773, -4.467), r = 0.293$	-	0.989821

表 12: 混合拟合统计指标

区间	拟合类型	MAE	RMSE	其他
$[a_1, a_2]$	左侧直线	0.001623	0.002130	斜率误差: 0.001882
	右侧直线	0.009615	0.011024	斜率误差: 0.010179
	中间圆弧	0.001104	0.001480	RSS: 0.002848
$[a_3, a_4]$	左侧直线	0.003380	0.004222	斜率误差: 0.004920
	右侧直线	0.000997	0.001278	斜率误差: 0.001356
	中间圆弧	0.002962	0.003335	RSS: 0.005248
$[a_5, a_6]$	左侧直线	0.004201	0.005151	斜率误差: 0.011666
	右侧直线	0.003522	0.004296	斜率误差: 0.009776
	中间圆弧	0.001419	0.001849	RSS: 0.002072

3.1.3 正式转折点的确定

由于初步确认的转折点依赖于人工干预, 无法精确地反映实际情况, 于是在分段拟合出曲线方程的基础上, 我们将拟合出的相邻的轮廓曲线联立得到交点作为模型的转折点

(包括水平直线), 从而得到对工件一水平轮廓线一般的数学模型。

选取的转折点具体坐标如下, 我们依次序从左到右对转折点进行编号。

表 13: 转折点坐标列表

点编号	a	b	点编号	a	b
1	49.772445	-1.767736	13	89.425324	-1.777153
2	52.685998	-1.766888	14	94.425123	-1.779506
3	57.661756	-1.767999	15	101.674392	-1.780994
4	59.692100	-1.767577	16	106.675598	-1.779732
5	64.721130	-1.768534	17	113.931704	-1.782479
6	66.780180	-1.765454	18	50.745317	-4.430017
7	71.795270	-1.769855	19	51.669307	-4.429409
8	76.807367	-0.774065	20	58.393964	-4.507349
9	81.809127	-1.777330	21	58.858080	-4.698625
10	84.806444	-1.778443	22	65.483778	-4.546202
11	86.608338	-1.773530	23	65.927967	-4.714993
12	87.606839	-1.782297			

3.1.4 参数值的确定

x_i : 对于水平线段 x_i , 直接利用选取的断点之差确定。

$\angle 1$: 对于角度 $\angle 1$, 设第一条斜线的方程为 $z = ax + b$, 则

$$\angle 1 = \pi - \arctan a$$

R_1 : 对于半径 R_1 则直接由圆的方程得出。

z_1 : 高度 z_1 由选取断点的高度之差确立。

c_1 : 圆心距离 c_1 由对应圆的方程确立。

表 14: 圆参数列表

圆编号	圆心坐标 (h,k)	半径 r	圆编号	圆心坐标 (h,k)	半径 r
O_1	(51.218281, -4.257183)	0.495084	O_5	(88.517301, -2.210691)	1.006212
O_2	(58.702350, -4.424920)	0.314907	O_6	(98.050106, -0.081529)	4.002952
O_3	(65.772786, -4.466850)	0.292671	O_7	(110.303008, -3.477862)	4.005216
O_4	(85.706230, -1.350278)	0.996464			

表 15: 转折点横坐标差值

区间	差值 Δx	区间	差值 Δx
(a_1, a_2)	2.914	(a_7, a_9)	10.014
(a_2, a_3)	4.976	(a_9, a_{10})	2.997
(a_3, a_4)	2.030	(a_{11}, a_{12})	0.999
(a_4, a_5)	5.029	(a_{13}, a_{14})	5.000
(a_5, a_6)	2.059	(a_{14}, a_{15})	7.249
(a_6, a_7)	5.015	(a_{15}, a_{16})	5.001
		(a_{16}, a_{17})	7.256

表 16: 圆心横坐标差值

区间	差值 Δc	区间	差值 Δc
(O_1, O_2)	7.484	(O_4, O_5)	2.811
(O_2, O_3)	7.070	(O_5, O_6)	9.533
(O_3, O_4)	19.933	(O_6, O_7)	12.253

参数 $z_1=1.00326554$

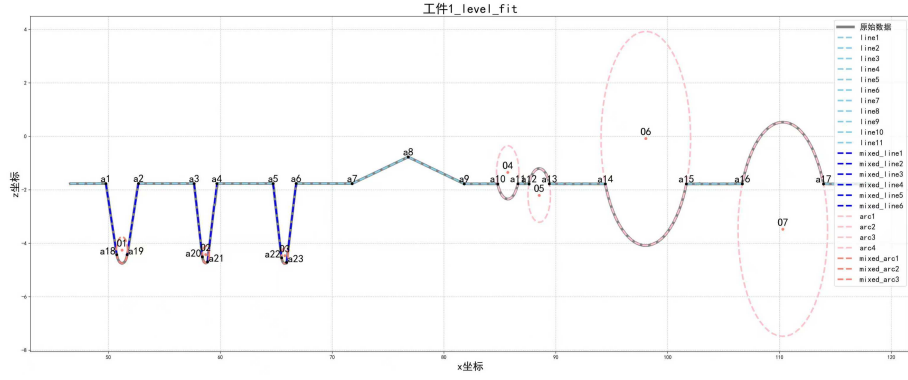


图 7: 拟合总图

表 17: 参数与数值对应表

参数	数值	参数	数值	参数	数值
x_1	2.914	x_7	10.014	$\angle 3$	104.965°
x_2	4.976	x_8	2.997	$\angle 4$	105.884°
x_3	2.030	x_9	0.999	$\angle 5$	105.353°
x_4	5.029	x_{10}	5.000	$\angle 6$	106.116°
x_5	2.059	x_{11}	7.249	$\angle 7$	168.763°
x_6	5.015	x_{12}	5.001	$\angle 8$	168.658°
l_1	2.835	x_{13}	7.256	l_5	2.881
l_2	2.850	$\angle 1$	110.074°	l_6	3.070
l_3	2.836	$\angle 2$	110.900°	l_7	5.110
l_4	3.047	z_1	1.003	l_8	5.101
R_1	0.495	R_5	1.006	c_3	19.933
R_2	0.315	R_6	4.003	c_4	2.811
R_3	0.293	R_7	4.005	c_5	9.533
R_4	0.996	c_1	7.484	c_6	12.253
		c_2	7.070		

3.2 倾角位移校正模型

我们将测量的数据做成图，如下：

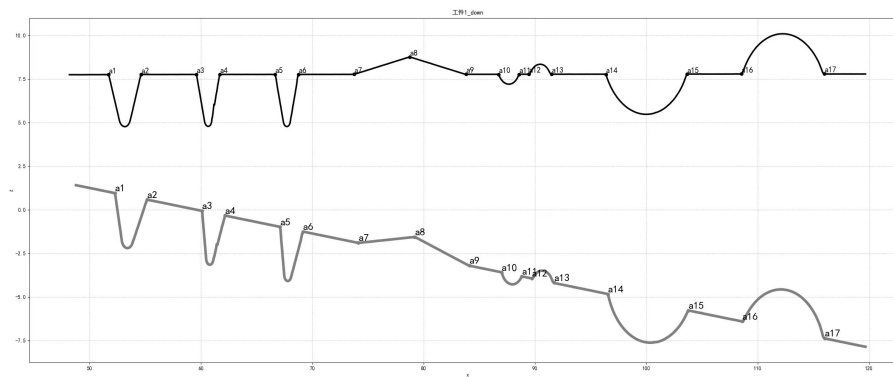


图 8

3.2.1 思路一

水平位移和偏转角度的确定 已知刚体的变换公式如下:

$$\begin{pmatrix} x' \\ z' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x \\ z \end{pmatrix} + \begin{pmatrix} \Delta x \\ \Delta z \end{pmatrix}$$

其中 θ 为旋转角度, $(\Delta x, \Delta z)$ 为位移量

所以接下来的目标即为确定最优的 $\theta, \Delta x, \Delta z$, 即从轮廓线坐标数据反推出工件的刚体运动参数, 即找到两组点云之间最优空间变换关系。

我们采用人工干预, 利用 Matlab 画出附件一 sheet2 的散点图, 选择与第一问中附件一 sheet1 对应的 17 个断点作为与其对应的初步特征点

表 18: 选取的 17 个转折点

No.	Position	No.	Position
1	[74.1437, -1.9026]	10	[84.1239, -3.2049]
2	[116.0250, -7.3731]	11	[79.2127, -1.5718]
3	[108.6285, -6.4093]	12	[69.1762, -1.2660]
4	[103.7796, -5.7907]	13	[67.1081, -0.9914]
5	[96.5639, -4.8494]	14	[62.1717, -0.3424]
6	[91.7218, -4.1919]	15	[60.0920, -0.0733]
7	[89.7228, -3.9415]	16	[55.1646, 0.5689]
8	[88.8237, -3.8364]	17	[52.2842, 0.9468]

接下页

No.	Position	No.	Position
9	[87.0161, -3.5982]		

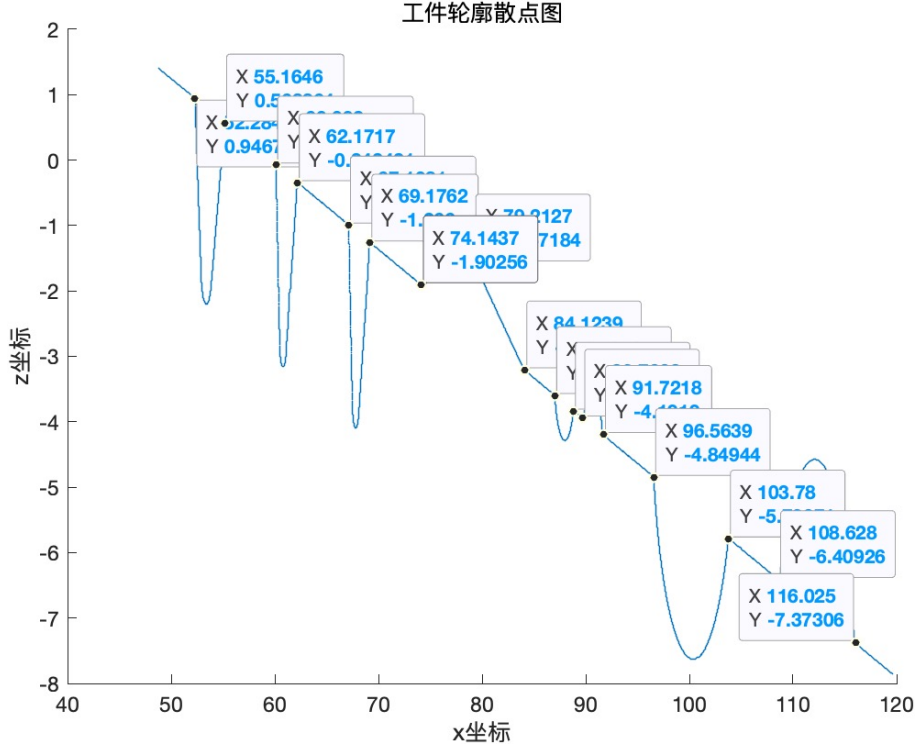


图 9

采用人工选择的特征点进行局部拟合，而后以拟合曲线的交点为最终特征点的方法进行偏转角度和水平位移的确认

参数确定 在两组数据中初步选取 17 个特征点后
利用与问题一相同的方法进行分区间拟合，将相邻拟合曲线的 17 个交点作为正式的特征点，利用最小二乘法，对两组正式的倾斜点进行偏转角和水平位移的计算

$$\begin{cases} x' = x \cos \theta - z \sin \theta + \Delta x \\ z' = x \sin \theta + z \cos \theta + \Delta z \end{cases}$$

在确立了 sheet1 的特征点集 $\{p_i\}$ ，sheet2 的特征点集 $\{q_i\} i \in \{1...n\}, n = 17$ 后，进行参数的确定

利用最小二乘法，目标函数为：

$$\min_{\theta, \Delta x} \sum_{i=1}^n [(x'_i - (x \cos \theta - z \sin \theta + \Delta x))^2 + (z'_i - (x \sin \theta + z \cos \theta + \Delta z))^2]$$

其中 x_i, x'_i, z_i, z'_i 为对应特征点的横纵坐标
利用 python 求得最优的变位参数

水平校正 在求得变位参数 $\theta, \Delta x$ 后, 利用反演变换:

$$\begin{pmatrix} x \\ z \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}^{-1} \begin{pmatrix} x' \\ z' \end{pmatrix} - \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}^{-1} \begin{pmatrix} \Delta x \\ 0 \end{pmatrix}$$

求得附件一 sheet2 的水平校正数据集, 带入问题一的模型即求得其对应的参数

3.2.2 思路二

偏转位移与角度确定 根据实验数据, 首先确定工件的水平位移和偏转角度。以原始数据为基础, 绘制带有偏转的散点图, 直观展示工件的初始状态。

基准线拟合与倾斜角计算 将水平方向作为基准线, 对采集的 17 组直线段数据进行线性拟合。采用一般式直线方程表示, 从而得到倾斜角度。

$$ax + by + c = 0$$

对全部 17 组直线夹角求算术平均值, 得到工件的整体倾斜角度:

$$\bar{\theta} = \frac{1}{17} \sum_{i=1}^{17} \theta_i$$

坐标旋转校正 通过坐标旋转将测量数据转换至水平基准系。设 (x_1, y_1) 为旋转后的观测坐标, (x, y) 为水平状态下的标准坐标, 转换关系为:

$$\begin{cases} x = x_1 \cos \bar{\theta} - y_1 \sin \bar{\theta} \\ y = y_1 \cos \bar{\theta} + x_1 \sin \bar{\theta} \end{cases}$$

水平状态参数求解 将校正后的坐标数据代入问题一的处理方法, 即可获得工件在水平状态下的精确几何参数。该转换过程消除了初始偏转带来的系统误差, 保证了后续分析的准确性。

3.2.3 最终模型采纳

最终, 我们选择方法二进行计算, 主要基于以下考虑:

- **方法一的局限性:**

- 特征点匹配对误差敏感: 依赖 17 个分段点的精确对应, 实际测量中工件倾斜可能导致局部形变, 影响最小二乘法的准确性。

- 全局拟合掩盖局部差异：通过单一参数描述整体位移时，可能忽略不同区段的偏转特性（如工件两端倾斜角度不一致）。
- **方法二的优势：**
 - 分段拟合更符合物理实际：通过独立计算各直线段偏转角，能更好反映工件的刚性分段旋转特性。
 - 更强的抗噪能力：对各段偏转角取平均可有效抑制局部数据波动的影响，结果更鲁棒。
 - 计算高效且透明：避免全局优化迭代，分步计算（先偏转校正后位移）流程更直观，便于验证。
- **特征点假设的合理性：**方法二通过分段几何特征（如直线斜率）隐含利用刚性特性，无需显式匹配特征点，减少误差传递。
- **效率：**我们认为思路二的计算量会比思路一的少。

综上，我们采用方法二作为最终解决方案。

问题二结果

全局参数

$$\text{偏转角度} = -7.4809 \quad (1)$$

直线段拟合结果

区间	拟合方程	R^2	p 值	角度	MAE	RMSE
$[, a_1]$	$z = 0.001524x + 7.668300$	0.4389	< 0.000001	179.91°	0.001072	0.001558
$[a_2, a_3]$	$z = 0.000595x + 7.720503$	0.3813	< 0.000001	179.97°	0.000778	0.001000
$[a_4, a_5]$	$z = 0.000059x + 7.754136$	0.0028	0.00000037	179.99°	0.001044	0.001460
$[a_6, a_7]$	$z = 0.000558x + 7.720063$	0.3167	< 0.000001	179.97°	0.000860	0.001090
$[a_7, a_8]$	$z = 0.200737x - 7.044889$	0.99998	< 0.000001	168.65°	0.001061	0.001296
$[a_8, a_9]$	$z = -0.199602x + 24.482357$	0.99998	< 0.000001	168.71°	0.000804	0.001097
$[a_9, a_{10}]$	$z = -0.000043x + 7.768713$	0.0010	0.027890	179.99°	0.000813	0.001005
$[a_{11}, a_{12}]$	$z = -0.003363x + 8.068299$	0.2613	< 0.000001	179.81°	0.000676	0.000825
$[a_{13}, a_{14}]$	$z = 0.000202x + 7.752312$	0.0398	< 0.000001	179.99°	0.000951	0.001284
$[a_{15}, a_{16}]$	$z = 0.000772x + 7.697689$	0.1738	< 0.000001	179.96°	0.001394	0.002182
$[a_{17},]$	$z = -0.000278x + 7.815536$	0.0413	< 0.000001	179.98°	0.001050	0.001274

圆弧拟合结果

区间	圆心坐标 (h, k)	半径 r	RSS	R^2	RMSE
$[a_{10}, a_{11}]$	(87.6578, 8.2007)	1.00231	0.005217	0.999749	0.001361
$[a_{12}, a_{13}]$	(90.4691, 7.3425)	0.99982	0.008891	0.999795	0.001670
$[a_{14}, a_{15}]$	(100.0005, 9.4796)	4.00894	0.076664	0.999981	0.002373
$[a_{16}, a_{17}]$	(112.2545, 6.0926)	3.99841	0.033301	0.999993	0.001546
$[a_1, a_2]_{\text{arc}}$	(53.1671, 5.2548)	0.49214	0.002494	0.998209	0.001413
$[a_3, a_4]_{\text{arc}}$	(60.6584, 5.1092)	0.32889	0.002253	0.973322	0.001954
$[a_5, a_6]_{\text{arc}}$	(67.7194, 5.0613)	0.29333	0.001828	0.991450	0.001616

混合区间拟合结果

区间	左侧直线	右侧直线	圆弧
	$z = -2.7269x + 148.7728$	$z = 2.7300x - 141.3504$	$O = (53.1671, 5.2548)$
$[a_1, a_2]$	$\theta = 110.14^\circ$	$\theta = 110.12^\circ$	$r = 0.49214$
	$R^2 = 0.99991$	$R^2 = 0.99958$	$R^2 = 0.99821$
	$z = -3.5927x + 221.8481$	$z = 3.9843x - 237.9242$	$O = (60.6584, 5.1092)$
$[a_3, a_4]$	$\theta = 105.55^\circ$	$\theta = 104.09^\circ$	$r = 0.32889$
	$R^2 = 0.99669$	$R^2 = 0.99204$	$R^2 = 0.97332$
	$z = -3.7400x + 257.1253$	$z = 3.6072x - 240.3185$	$O = (67.7194, 5.0613)$
$[a_5, a_6]$	$\theta = 104.97^\circ$	$\theta = 105.49^\circ$	$r = 0.29333$
	$R^2 = 0.99608$	$R^2 = 0.98866$	$R^2 = 0.99145$

完整转折点坐标

表 23: 全部转折点坐标

转折点	坐标 (a_i, b_i)
a_1	(51.71612752, 7.74713400)
a_2	(54.61731276, 7.75299316)
a_3	(59.59042966, 7.75595150)
a_4	(61.66283071, 7.75777330)
a_5	(66.67629630, 7.75806905)
a_6	(68.77197985, 7.75842272)

续表：全部转折点坐标

转折点	坐标 (a_i, b_i)
a_7	(73.75861804, 7.76120415)
a_8	(78.75129234, 8.76341921)
a_9	(83.75279007, 7.76510882)
a_{10}	(86.75512121, 7.76497963)
a_{11}	(88.56307010, 7.77048567)
a_{12}	(89.56389253, 7.76712019)
a_{13}	(91.37253129, 7.77076049)
a_{14}	(96.37349704, 7.77177023)
a_{15}	(103.63023566, 7.77772688)
a_{16}	(108.63030918, 7.78158866)
a_{17}	(115.87780736, 7.78333038)
a_{18}	(52.69376049, 5.08120920)
a_{19}	(53.63891076, 5.08198905)
a_{20}	(60.35094653, 5.02362222)
a_{21}	(60.97794371, 5.02899191)
a_{22}	(67.41885266, 4.98093164)
a_{23}	(68.00251866, 4.98279241)

完整几何参数

表 24: 全部关键几何参数

参数	值
$x_1 = a_2 - a_1$	2.901 185 24
$x_2 = a_3 - a_2$	4.973 116 90
$x_3 = a_4 - a_3$	2.072 401 05
$x_4 = a_5 - a_4$	5.013 465 59
$x_5 = a_6 - a_5$	2.095 683 55
$x_6 = a_7 - a_6$	4.986 638 19
$x_7 = a_8 - a_7$	9.994 172 03
$x_8 = a_{10} - a_9$	3.002 331 14
$x_9 = a_{12} - a_{11}$	1.000 822 43
$x_{10} = a_{14} - a_{13}$	5.000 965 75
$x_{11} = a_{15} - a_{14}$	7.256 738 62

续表：全部关键几何参数

参数	值
$x_{12} = a_{16} - a_{15}$	5.000 073 52
$x_{13} = a_{17} - a_{16}$	7.247 498 18
$c_1 = O_2^x - O_1^x$	7.491 253 45
$c_2 = O_3^x - O_2^x$	7.060 985 49
$c_3 = O_4^x - O_3^x$	19.938 402 63
$c_4 = O_5^x - O_4^x$	2.811 293 06
$c_5 = O_6^x - O_5^x$	9.531 396 77
$c_6 = O_7^x - O_6^x$	12.253 997 44
z_1	0.998 310 39

完整圆心参数

表 25: 全部圆心参数

圆心	坐标 (h, k)	半径 r
O_1	(53.16713554, 5.25480264)	0.492 139 69
O_2	(60.65838899, 5.10919580)	0.328 891 47
O_3	(67.71937448, 5.06128574)	0.293 333 78
O_4	(87.65777711, 8.20068677)	1.002 311 52
O_5	(90.46907017, 7.34252163)	0.999 815 23
O_6	(100.00046694, 9.47959528)	4.008 937 14
O_7	(112.25446438, 6.09258206)	3.998 405 21

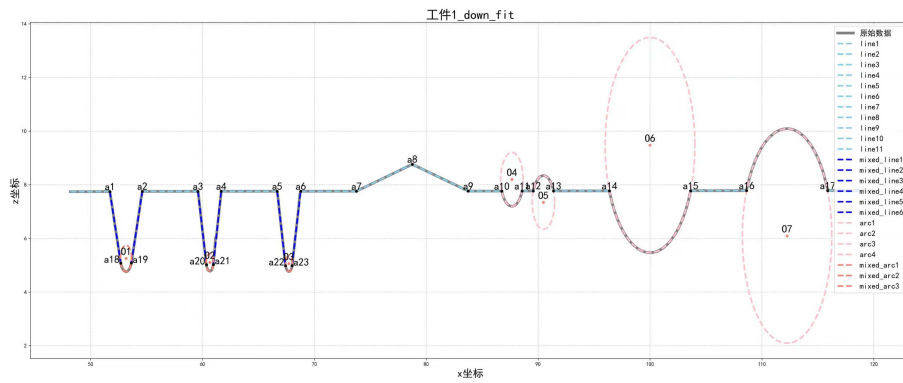


图 10: 拟合总图

表 26: 参数与数值对应表

参数	数值	参数	数值	参数	数值
x_1	2.901	x_7	9.994	$\angle 3$	105.554°
x_2	4.973	x_8	3.002	$\angle 4$	104.089°
x_3	2.072	x_9	1.000	$\angle 5$	104.970°
x_4	5.013	x_{10}	5.001	$\angle 6$	105.495°
x_5	2.096	x_{11}	7.257	$\angle 7$	168.650°
x_6	4.987	x_{12}	5.000	$\angle 8$	168.712°
l_1	2.840	x_{13}	7.247	l_5	2.878
l_2	2.845	$\angle 1$	110.139°	l_6	2.880
l_3	2.836	$\angle 2$	110.118°	l_7	5.029
l_4	2.813	z_1	0.998	l_8	5.100
R_1	0.492	R_5	1.000	c_3	19.938
R_2	0.329	R_6	4.009	c_4	2.811
R_3	0.293	R_7	4.000	c_5	9.531
R_4	1.002	c_1	7.491	c_6	12.254
		c_2	7.061		

两次数数据的比较 我们可以计算并列第二次数据减去第一次数据的差值，并且用差值除以第一次的数据，得到变化率。

表 27: 数值差异与相对变化率对比 (第二次 - 第一次)

参数	差值 (mm)	变化率 (%)	参数	差值 (mm)	变化率 (%)
x_1	-0.013	-0.45	x_8	0.005	0.17
x_2	-0.003	-0.06	x_9	0.001	0.10
x_3	0.042	+2.07	x_{10}	0.001	0.02
x_4	-0.016	-0.32	x_{11}	0.008	0.11
x_5	0.037	+1.80	x_{12}	-0.001	-0.02
x_6	-0.028	-0.56	x_{13}	-0.009	-0.12
x_7	-0.020	-0.20	$\angle 1$	0.065	0.06
$\angle 2$	-0.782	-0.71	l_1	0.005	0.18
$\angle 3$	0.589	+0.56	l_2	-0.005	-0.18
$\angle 4$	-1.795	-1.70	l_3	0.000	0.00
$\angle 5$	-0.383	-0.36	l_4	-0.234	-7.68
$\angle 6$	-0.621	-0.59	l_5	-0.003	-0.10
$\angle 7$	-0.113	-0.07	l_6	-0.190	-6.19
$\angle 8$	0.054	0.03	l_7	-0.081	-1.59
z_1	0.000	0.00	l_8	-0.001	-0.02
R_1	-0.003	-0.61	R_5	-0.006	-0.60
R_2	0.014	+4.44	R_6	0.006	0.15
R_3	0.000	0.00	R_7	-0.005	-0.12
R_4	0.006	0.60	c_1	0.007	0.09
c_2	-0.009	-0.13	c_3	0.005	0.03
c_4	0.000	0.00	c_5	-0.002	-0.02
c_6	0.001	0.01			

3.3 问题三求解

3.3.1 基准线选择与测量原理

我们将十次测量的数据做成图如下：

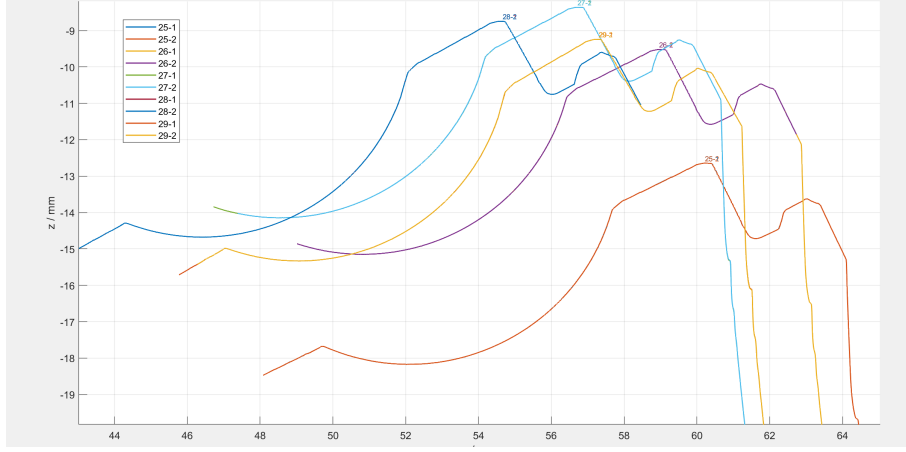


图 11

在工件 2 的轮廓测量中，我们选择第三段直线作为工艺基准面进行倾斜角度计算。该基准面的选取基于以下考虑：

- 几何特性：该段为工件轮廓中最长的直线段。
- 稳定性：直线特征明显，受测量噪声影响较小。
- 代表性：能够反映工件的整体倾斜状态。

3.3.2 单次测量数学模型

对于第 i 次测量获得的轮廓数据，定义第三段直线的点集为：

$$\mathcal{P}_i = \{(x_{ij}, z_{ij})\}_{j=1}^{N_i}$$

其中 N_i 表示第 i 次测量的数据点数量。

采用最小二乘法拟合直线方程：

$$z = k_i x + b_i$$

拟合参数计算如下：

$$k_i = \frac{N_i \sum_{j=1}^{N_i} x_{ij} z_{ij} - \left(\sum_{j=1}^{N_i} x_{ij} \right) \left(\sum_{j=1}^{N_i} z_{ij} \right)}{N_i \sum_{j=1}^{N_i} x_{ij}^2 - \left(\sum_{j=1}^{N_i} x_{ij} \right)^2}, \quad b_i = \frac{\sum_{j=1}^{N_i} z_{ij} - k_i \sum_{j=1}^{N_i} x_{ij}}{N_i}$$

据此得到第 i 次测量的倾斜角度：

$$\alpha_i = \arctan(k_i) \times \frac{180}{\pi}$$

3.3.3 基准角度建立与偏差分析

为消除测量系统误差，建立标准参考角度：

$$\alpha_{\text{ref}} = \frac{1}{10} \sum_{i=1}^{10} \alpha_i$$

定义第 i 次测量的相对角度偏差：

$$\Delta\alpha_i = \alpha_{\text{ref}} - \alpha_i$$

基准选择合理性 采用多次测量平均值作为基准角度，可有效消除以下影响：

- 测量平台的水平度误差
- 单次测量的随机误差
- 工件放置的微小偏差

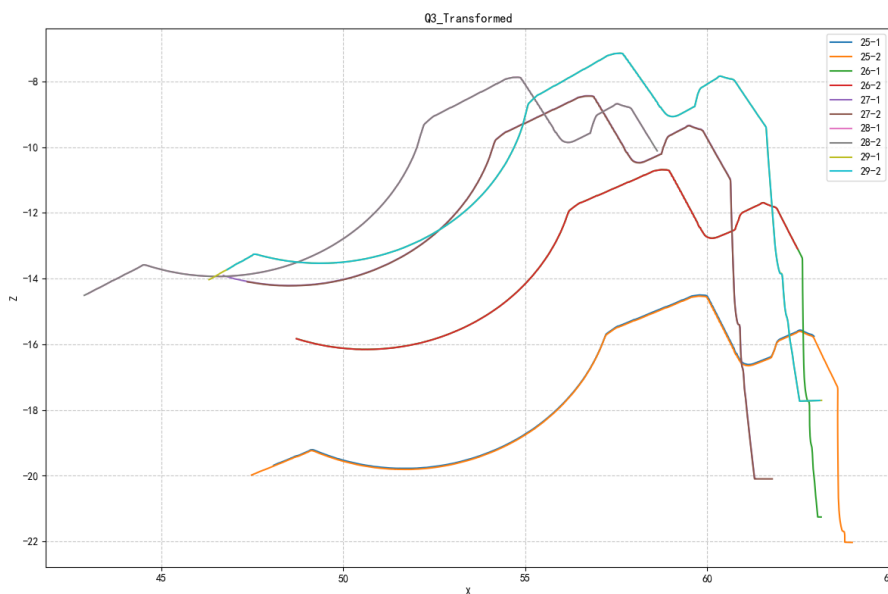


图 12

3.3.4 计算水平偏移量

再选取目标直线右侧与之相交的那条直线，目测其区间，并拟合，得到 10 个交点。取 10 个交点的质心为标准点，计算各自的水平偏差后，把 10 份旋转后的数据平移到标准点，再对 z 轴进行平均，得到轮廓图，并根据该图计算参数。

这是旋转并对 x 平移后的图：

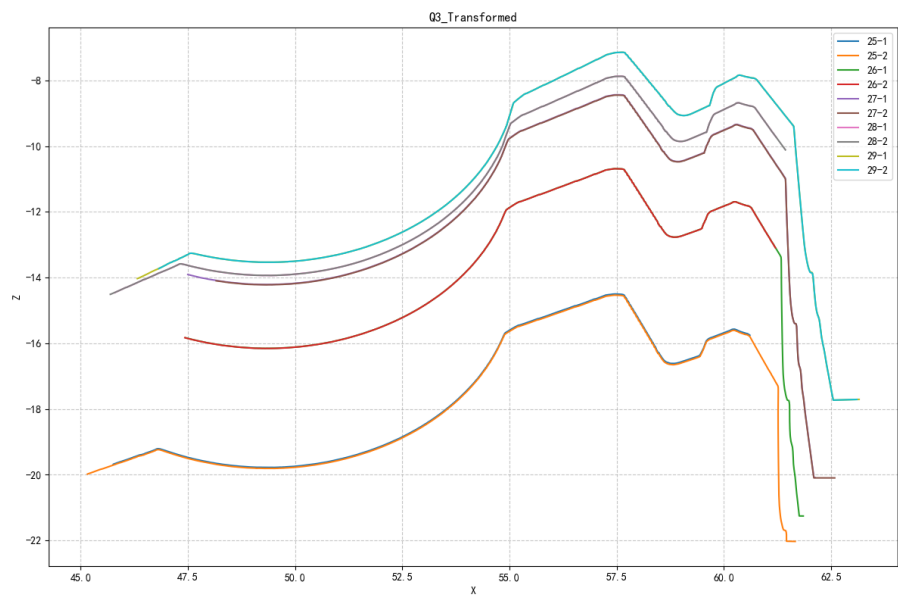


图 13

这是进一步对 z 平均后的图，也是后续拟合并计算参数的基础：

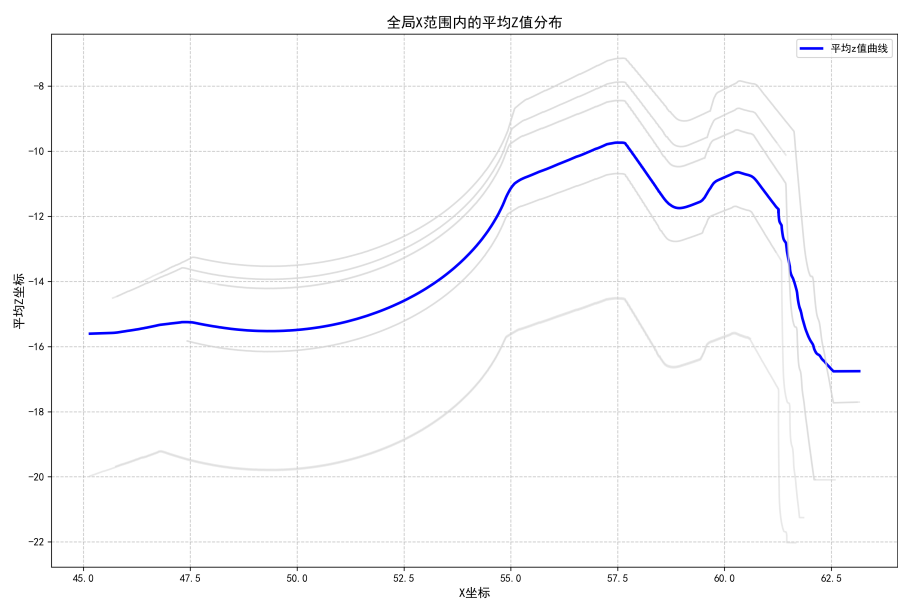


图 14

3.3.5 结果

表 28: 原始角度测量结果

测量编号	角度 (弧度)	角度 (度)
25-1	0.455	26.081
25-2	0.455	26.047
26-1	0.466	26.718
26-2	0.466	26.715
27-1	0.485	27.776
27-2	0.485	27.765
28-1	0.502	28.768
28-2	0.502	28.768
29-1	0.523	29.946
29-2	0.523	29.946

表 29: 相对角度偏差

测量编号	相对角度 (弧度)	相对角度 (度)
25-1	-0.031	-1.772
25-2	-0.032	-1.806
26-1	-0.020	-1.135
26-2	-0.020	-1.138
27-1	-0.001	-0.077
27-2	-0.002	-0.088
28-1	0.016	0.915
28-2	0.016	0.915
29-1	0.037	2.093
29-2	0.037	2.093

我们可以得到 Q3 拟合图

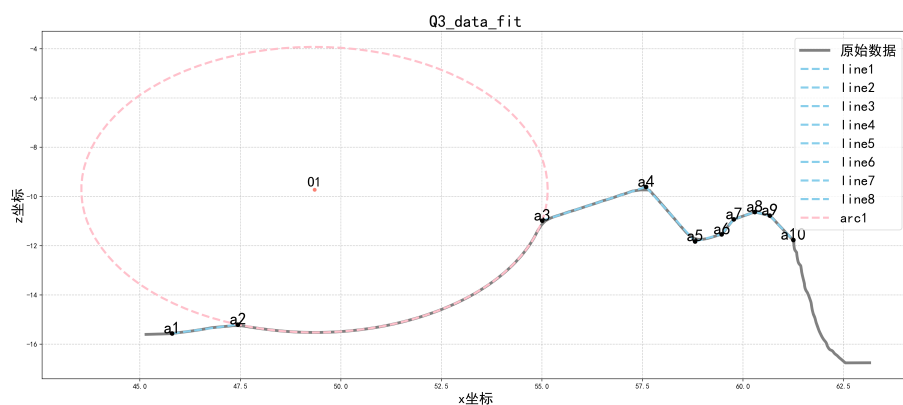


图 15

表 30: 区间拟合方程

区间	拟合方程
$[a_1, a_2]$	$z = 0.222x - 25.753$
$[a_2, a_3]$	圆弧 $(49.345, -9.726)$, $r = 5.800$
$[a_3, a_4]$	$z = 0.532x - 40.238$
$[a_4, a_5]$	$z = -1.811x + 94.703$
$[a_5, a_6]$	$z = 0.435x - 37.400$
$[a_6, a_7]$	$z = 2.005x - 130.765$
$[a_7, a_8]$	$z = 0.575x - 45.276$
$[a_8, a_9]$	$z = -0.385x + 12.588$
$[a_9, a_{10}]$	$z = -1.720x + 93.556$

表 31: 拟合统计指标

区间	R^2	p 值	斜率误差	MAE	RMSE	角度 ($^\circ$)	弧度
$[a_1, a_2]$	0.993 000	0.000 000	0.000 468	0.005 902	0.007 450	167.464	2.9228
$[a_2, a_3]$	1.000 000	—	—	0.001 075	0.001 780	—	—
$[a_3, a_4]$	1.000 000	0.000 000	0.000 104	0.002 291	0.002 821	151.995	2.6528
$[a_4, a_5]$	0.998 000	0.000 000	0.002 199	0.020 672	0.027 816	118.901	2.0752
$[a_5, a_6]$	0.969 000	0.000 000	0.003 058	0.010 295	0.012 962	156.498	2.7314
$[a_6, a_7]$	0.996 000	0.000 000	0.007 305	0.008 464	0.010 424	116.510	2.0335
$[a_7, a_8]$	0.999 000	0.000 000	0.000 711	0.001 309	0.002 268	150.119	2.6201
$[a_8, a_9]$	0.997 000	0.000 000	0.001 167	0.001 505	0.002 126	158.937	2.7740
$[a_9, a_{10}]$	1.000 000	0.000 000	0.001 345	0.003 788	0.005 716	120.177	2.0975

表 32: 转折点坐标

点名称	x 坐标	y 坐标
(a1,b1)	45.800 080 04	-15.566 277 85
(a2,b2)	47.439 195 07	-15.204 169 34
(a3,b3)	55.006 780 84	-10.984 639 53
(a4,b4)	57.587 193 42	-9.612 332 38
(a5,b5)	58.809 505 09	-11.826 476 64
(a6,b6)	59.470 006 60	-11.539 258 08
(a7,b7)	59.773 187 03	-10.931 442 74
(a8,b8)	60.293 651 09	-10.632 393 15
(a9,b9)	60.667 863 36	-10.776 510 63
(a10,b10)	61.249 808 54	-11.764 127 89

表 33: 圆弧几何参数

圆心	x 坐标	y 坐标	半径
O1	49.344 923 66	-9.726 158 86	5.800 034 51

圆弧长度 (a2,a3)=9.78390252

表 34: 轮廓片段长度统计

类型	区间	长度
直线段	(a1,a2)	1.678 636 55
直线段	(a3,a4)	2.922 628 27
直线段	(a4,a5)	2.529 126 45
直线段	(a5,a6)	0.720 247 70
直线段	(a6,a7)	0.679 233 29
直线段	(a7,a8)	0.600 261 19
直线段	(a8,a9)	0.401 004 57
直线段	(a9,a10)	1.146 319 34
圆弧段	(a2,a3)	9.783 902 52

Q3 轮廓及部分参数标记:

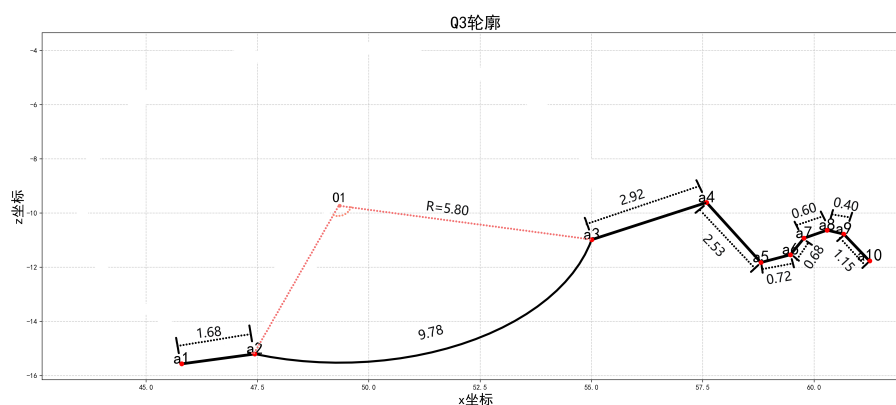


图 16

3.4 问题四

3.4.1 圆的修正

附件 3 提供了工件 2 关于圆 9 次局部测量数据，我们将这些数据画成图如下，再线性拟合，与前面的做法类似。

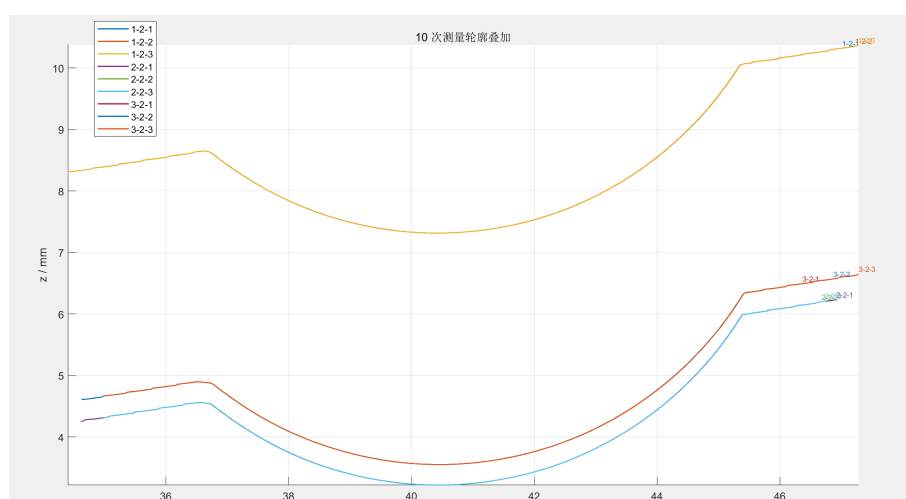


图 17

我们可以找到最低点，从而把最低点记成特征点。由于只需要对工件 2 的轮廓校正，即我们需要计算工件 2 圆角部分的参数即可：即圆半径和弧长。将附件三中的数据经过上下左右平移，使特征点重合。

得到结果。

3.4.2 角的修正

附件 4 提供了工件 2 关于角的 9 次局部测量数据，我们将这些数据画成图如下。

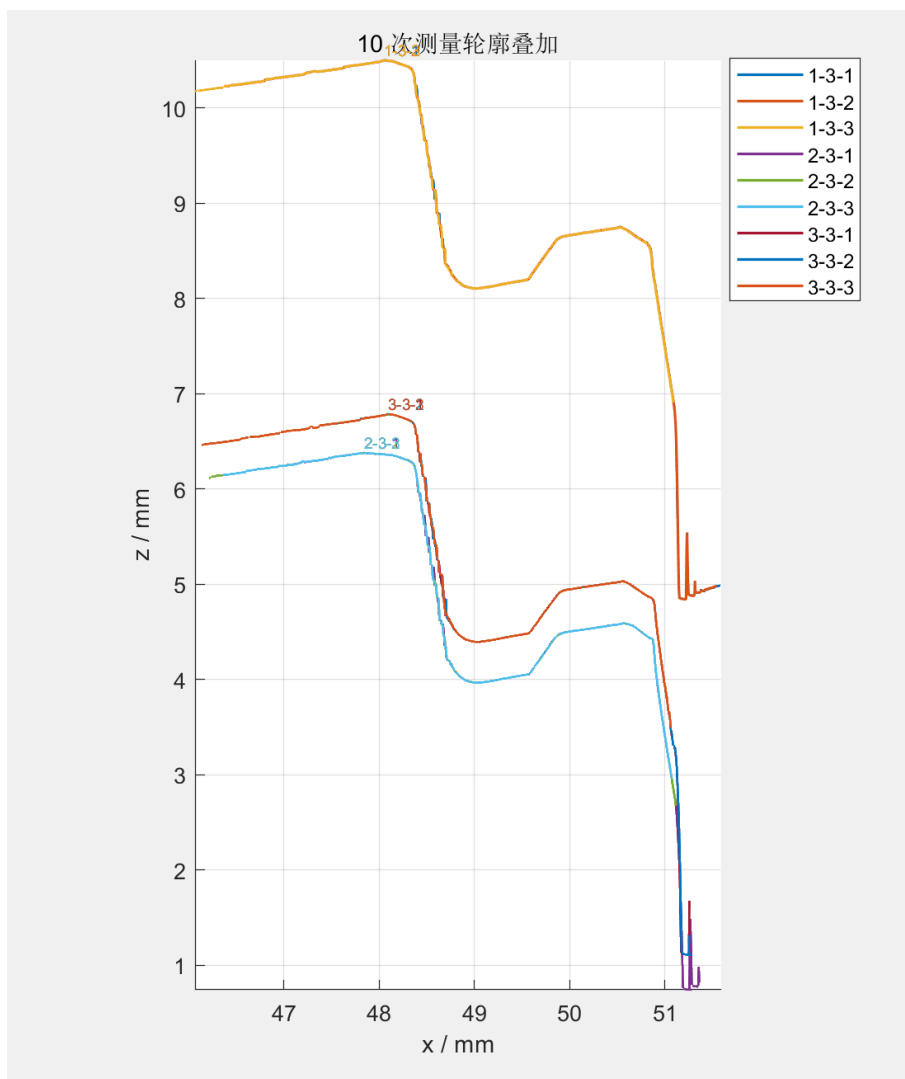


图 18

令最高点为特征点，通过平移令特征点重合，再线性拟合，与前面的做法类似。得到结果。

4 总结分析及应用

本文建立的轮廓线参数计算模型通过多阶段处理方法实现了精确的几何特征提取，其主要特点包括：

- 分阶段建模框架：
 - 采用曲率分析法自动定位断点，避免人工选取的主观性
 - 通过预处理去除孤立离群点，提高数据质量
- 鲁棒拟合算法：

- 直线段采用 RANSAC 方法进行拟合
- 强制直线与圆弧在过渡点保持连续
- **实际应用效果：**
 - 为轮廓配准和误差分析建立统一坐标系
 - 通过相对角度转换消除系统误差

5 模型评价与推广

5.1 模型评价

我们的方法在数据拟合、误差校正、标准化处理和数据融合等方面均取得了良好的效果，具有较强的鲁棒性和实用性。

我们在分段最小二乘拟合与解析几何结合的方法下，成功建立了工件 1 轮廓线的精确数学模型，准确识别了转折点坐标及相关参数，确保了轮廓线拟合的精度。

我们在倾斜数据校正的基准线处理方法下，通过水平校正和整体最小二乘拟合，显著提高了工件在倾斜状态下的参数估计精度，有效降低了测量误差。

我们在以最长直线段为基准的标准化处理策略下，通过计算相对倾斜角并求取平均值，消除了系统误差，建立了稳定的参考角度体系，使参数估计更加可靠。

我们在基于特征点匹配的精度提升算法下，通过多组数据的空间对齐与融合拟合，进一步提高了局部测量数据的准确性，为类似工件的几何参数测量提供了可推广的技术方案。

5.2 推广潜力

本方法体系具有以下可扩展性：

- **数据维度扩展：**
 - 给定更高采样频率 f_s ，可改进曲率计算为：

$$k(t) = \frac{\|\dot{\mathbf{r}}(t) \times \ddot{\mathbf{r}}(t)\|}{\|\dot{\mathbf{r}}(t)\|^3} \quad (2)$$

- 增加正交视角数据后，可构建三维曲率场：

$$\kappa(x, y, z) = \nabla \times (\nabla \times \mathbf{r}) \quad (3)$$

- **实时处理：**若配备 $\Delta t \leq 10ms$ 的实时数据流，可建立增量式更新：

$$\hat{\theta}_t = \alpha \theta_{t-1} + (1 - \alpha) \theta_{new} \quad (4)$$

- **约束增强：**引入 C^2 连续性约束后，可处理更复杂的过渡特征：

$$\min \sum \|r_i - f(s_i)\|^2 + \lambda \|\nabla^2 f\|^2 \quad (5)$$

参考文献

- [1] 戴静兰, 陈志杨, 叶修梓. ICP 算法在点云配准中的应用 [J]. 中国图象图形学报, 2007, 12(3): 517–521.
- [2] 仇谷烽, 郭培基, 谢滨, 等. 接触式非球面轮廓测量的数据处理模型 [J]. 光学精密工程, 2007, 15(4): 492–498.
- [3] 韩志国, 李锁印, 冯亚南, 等. 接触式轮廓仪探针状态检查图形样块的研制 [J]. 微纳电子技术, 2019, 56(9): 761–765.
- [4] 颜鲁林. 接触式轮廓仪自动标注方法研究 [J]. 甘肃高师学报, 2021, 26(2): 36–40.
- [5] Supaporn Jaturunruangsri. Evaluation of Material Surface Profiling Methods: Contact versus Non-contact[D]. Brunel University London, 2014.
- [6] 李美霞. 接触式轮廓仪的自动标注问题分析 [J]. 延安职业技术学院学报, 2023, 37(1): 103–108.
- [7] 王阿利. 工件轮廓线参数校正方法研究 [J]. 内燃机与配件, 2021(10): 58–59.
- [8] 付嵌波. 触针式轮廓仪几何误差补偿算法研究与实验 [D]. 西安: 西安工业大学, 2023.
- [9] 薛毅. ”接触式轮廓仪的自动标注”的问题解析 [J]. 数学建模及其应用, 2021, 10(1): 92–100.
- [10] 高彤, 陈鸿, 张亮, 等. 接触式轮廓扫描仪系统标定算法 [J]. 电子技术应用, 2021, 47(3): 65–70+74.
- [11] 蒋培军. 接触式轮廓仪测量的自动标注问题探究——基于最小二乘法 [J]. 南通职业大学学报, 2023, 37(3): 66–70.
- [12] 张穗, 刘楠. 接触式轮廓仪的测量精度问题 [J]. 理论算法, 2022(8): 55–57.
- [13] 喻雪. 接触式轮廓仪的工件形状自动标注数学模型研究 [J]. 兰州石化职业技术学院学报, 2021, 21(2): 15–17.
- [14] 乔旭安, 陈丽莎, 李华, 等. 接触式轮廓仪的自动标注参数及分析 [J]. 科技与创新, 2021(7): 97–100 .
- [15] 王云庆, 李庆祥, 周兆英. 接触式轮廓测量中触针测量力的分析 [J]. 现代计量测试, 1996(01): 18–21+17.
- [16] 伍春兰. 直线方程、倾斜角与斜率的教学实践与反思 [J]. 数学通报, 2017, 56(05): 30–33+63.

附录

第一问代码

```
1
2 #This is a file for Q1.
3 import numpy as np
4 from numpy import argmin
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import math
8 from scipy import stats
9 from scipy.optimize import least_squares
10
11 import os
12
13 #设置字体
14 plt.rcParams['font.sans-serif'] = ['SimHei']
15 #设置负号
16 plt.rcParams['axes.unicode_minus'] = False
17
18 os.makedirs('Figures', exist_ok=True)
19
20 #区间数据读取器
21 def data_reader(start,end,df):
22     # 读取数据
23     x = df['x']
24     #选取区间内部的点
25     start_idx = np.searchsorted(x, start+0.2, side='left')
26     end_idx = np.searchsorted(x, end-0.2, side='right')
27     x = df['x'].iloc[start_idx:end_idx + 1].values
28     z = df['z'].iloc[start_idx:end_idx + 1].values
29     return x,z
30
31 #角度计算器（大角）
32 def angle_cal(slope):
33     if slope >= 0:
34         angle_rad = np.pi-math.atan(slope) # 弧度
35         angle_deg = math.degrees(angle_rad) # 角度
36     else:
```

```

37     angle_rad = np.pi+math.atan(slope)
38     angle_deg = math.degrees(angle_rad)
39     return angle_rad,angle_deg
40
41 #交点计算器
42 def line_intersection(line1, line2, tol=1e-6):
43     """
44     计算直线间的交点
45     :param: 直线参数 (Ai, Bi, Ci)=(slope,-1,intercept) 表示  $A_i \cdot x + B_i \cdot z + C_i = 0$ 
46     """
47     A1, B1, C1 = line1
48     A2, B2, C2 = line2
49     # 计算行列式
50     det = A1 * B2 - A2 * B1
51     # 处理特殊情况
52     if abs(det) < tol:
53         # 检查是否重合
54         if abs(A1 * C2 - A2 * C1) < tol and abs(B1 * C2 - B2 * C1) < tol:
55             return "直线重合"
56         else:
57             return "直线平行"
58     # 计算交点坐标
59     x = (B1 * C2 - B2 * C1) / det
60     z = (A2 * C1 - A1 * C2) / det
61     return (x, z)
62
63 def line_circle_intersection(line, circle, tol=0.7):
64     """
65     计算直线与圆的交点
66     :param line: 直线参数 (A, B, C)=(slope,-1,intercept) 表示  $Ax + Bz + C = 0$ 
67     :param circle: 圆参数 (h, k, r) 圆心(h,k)和半径r
68     """
69     A, B, C = line
70     h, k, r = circle
71     # 特殊情况处理: 垂直线 (B=0)
72     if abs(B) < tol:
73         # 垂直线方程:  $Ax + C = 0 \rightarrow x = -C/A$ 

```

```

74     x0 = -C / A
75     # 代入圆方程: (x0-h)^2 + (z-k)^2 = r^2
76     d_sq = r ** 2 - (x0 - h) ** 2
77     if d_sq < -tol:
78         print("无交点")
79         return () # 无交点
80     elif abs(d_sq) < tol:
81         print("相切")
82         return (x0, k) # 相切
83     else:
84         y1 = k + np.sqrt(d_sq)
85         y2 = k - np.sqrt(d_sq)
86         print("两个交点")
87         return (x0, y1), (x0, y2)
88     # 将直线表示为 z = mx + c
89     m = -A / B
90     c = -C / B
91     # 展开为二次方程: ax^2 + bx + c = 0
92     a = 1 + m ** 2
93     b = 2 * (m * (c - k) - h)
94     c_eq = h ** 2 + (c - k) ** 2 - r ** 2
95     # 计算判别式
96     delta = b ** 2 - 4 * a * c_eq
97     print('delta=', delta)
98     if delta < -tol:
99         print("无交点")
100        return () # 无交点
101    elif abs(delta) < tol:
102        print("相切")
103        x = -b / (2 * a)
104        y = m * x + c
105        return (x, y)
106    else:
107        # 两个交点
108        print("两个交点")
109        sqrt_disc = np.sqrt(delta)
110        x2 = (-b + sqrt_disc) / (2 * a)
111        x1 = (-b - sqrt_disc) / (2 * a)
112        return (x1, m * x1 + c), (x2, m * x2 + c)
113

```

```

114 #直线拟合器
115 def linear_fit(start,end,df):
116     # 读取数据
117     x,z=data_reader(start,end,df)
118
119     # 计算线性拟合参数
120     slope, intercept, r_value, p_value, std_err =
121         stats.linregress(x, z)
122     angle_rad,angle_deg=angle_cal(slope)
123
124     # 计算评价指标
125     y_pred = slope * x + intercept
126     residuals = z - y_pred
127     mae = np.mean(np.abs(residuals)) # 平均绝对误差
128     rmse = np.sqrt(np.mean(residuals ** 2)) # 均方根误差
129
130     print(f"拟合直线方程: z = {slope:.8f}x + {intercept:.8f}\n"
131           f"R^2: {r_value ** 2:.8f}\n"
132           f"p值: {p_value:.8f}\n"
133           f"斜率标准误差: {std_err:.8f}\n"
134           f"MAE: {mae:.8f}\n"
135           f"RMSE: {rmse:.8f}\n"
136           f"直线角度: {angle_deg:.4f}° (弧度: {angle_rad:.4f})\n"
137           )
138
139     return (slope,-1,intercept)
140
141 #弧线拟合器
142 def arc_fit(start,end,df):
143     # 读取数据
144     x,z=data_reader(start,end,df)
145
146     # 初始参数估计
147     # 使用数据的质心作为圆心初始估计
148     h0 = np.mean(x)
149     k0 = np.mean(z)
150
151     # 计算到质心的平均距离作为半径初始估计
152     r0 = np.sqrt(np.mean((x - h0) ** 2 + (z - k0) ** 2))
153
154     #定义残差
155     def residuals(params):
156         h, k, r = params

```



```

153         return np.sqrt((x - h) ** 2 + (z - k) ** 2) - r
154
155     # 使用LM算法最小化残差平方和
156     result = least_squares(
157         fun=residuals, # 残差函数
158         x0=[h0, k0, r0], # 初始参数
159         method='lm', # Levenberg-Marquardt方法
160         ftol=1e-8, # 函数容忍度
161         xtol=1e-8, # 参数容忍度
162         max_nfev=100 # 最大函数评估次数
163     )
164     h_fit, k_fit, r_fit = result.x
165     # 确保半径为正值
166     r_fit = abs(r_fit)
167
168     # 计算评价指标
169     res = residuals((h_fit, k_fit, r_fit))
170     rss = np.sum(res ** 2) # 残差平方和
171     sst = np.sum((z - np.mean(z)) ** 2) # 总平方和
172     r_squared = 1 - (rss / sst) # 决定系数
173     mae = np.mean(np.abs(res)) # 平均绝对误差
174     rmse = np.sqrt(np.mean(res ** 2)) # 均方根误差
175
176     print(f"拟合参数(h, k, r)={(h_fit, k_fit, r_fit)}\n"
177           f"残差平方和(RSS): {rss:.8f}\n"
178           f"R^2: {r_squared:.8f}\n"
179           f"MAE: {mae:.8f}\n"
180           f"RMSE: {rmse:.8f}\n")
181     return (h_fit, k_fit, r_fit)
182
183 #混合拟合器
184 def mixed_fit(start, end, target_z1, target_z2, target_z3, df):
185     #获取区间
186     x, z = data_reader(start, (start + end) / 2, df)
187     # 获取原始数据的全局索引
188     start_idx = np.searchsorted(df['x'], start + 0.2, side='left')
189     idx1 = start_idx + np.argmin(np.abs(z - target_z1))
190     idx2 = start_idx + np.argmin(np.abs(z - target_z2))
191     idx3 = start_idx + np.argmin(np.abs(z - target_z3))
192

```

```

193 x, z = data_reader((start + end) / 2, end, df)
194 mid_idx = np.searchsorted(df['x'], (start+end)/2+0.2,
195                             side='left')
196 idx6 = mid_idx + np.argmin(np.abs(z - target_z1))
197 idx5 = mid_idx + np.argmin(np.abs(z - target_z2))
198 idx4 = mid_idx + np.argmin(np.abs(z - target_z3))
199
200 #左侧直线拟合
201 print(f"左侧直线区间的拟合结果: ")
202 mixed_line1=linear_fit(df['x'][idx1],df['x'][idx2],df)
203
204 #右侧直线拟合
205 print(f"右侧直线区间的拟合结果: ")
206 mixed_line2=linear_fit(df['x'][idx5],df['x'][idx6],df)
207
208 #中间弧线拟合
209 print(f"中间弧线区间的拟合结果: ")
210 mixed_arc=arc_fit(df['x'][idx3],df['x'][idx4],df)
211
212 return mixed_line1,mixed_line2,mixed_arc
213
214 def plot_all_results(df):
215     plt.figure(figsize=(24, 10))
216
217     # 1. 绘制原始数据
218     plt.plot(df['x'], df['z'], color='gray', linewidth=5,
219             label='原始数据')
220
221     # 2. 绘制所有直线段拟合曲线
222     def plot_fitted_line(line,start,end,line_name,color='skyblue'):
223         slope, _, intercept = line
224         x_fit = np.linspace(start, end, 100)
225         y_fit = slope * x_fit + intercept
226         plt.plot(x_fit, y_fit, color=color, linestyle='--',
227                 linewidth=3, label=line_name)
228
229     plot_fitted_line(line1,df['x'][0],turning_dict['a1'],'line1')
230     plot_fitted_line(line2,turning_dict['a2'],turning_dict['a3'],'line2')
231     plot_fitted_line(line3,turning_dict['a4'],turning_dict['a5'],'line3')
232     plot_fitted_line(line4,turning_dict['a6'],turning_dict['a7'],'line4')

```

```

230 plot_fitted_line(line5,turning_dict['a7'],turning_dict['a8'],'line5')
231 plot_fitted_line(line6,turning_dict['a8'],turning_dict['a9'],'line6')
232 plot_fitted_line(line7,turning_dict['a9'],turning_dict['a10'],'line7')
233 plot_fitted_line(line8,turning_dict['a11'],turning_dict['a12'],'line8')
234 plot_fitted_line(line9,turning_dict['a13'],turning_dict['a14'],'line9')
235 plot_fitted_line(line10,turning_dict['a15'],turning_dict['a16'],'line10')
236 plot_fitted_line(line11,turning_dict['a17'],df['x'].iloc[-1],'line11')
237
238 plot_fitted_line(mixed_line1,turning_dict['a1'],turning_dict['a18'],'mixed_1
239 plot_fitted_line(mixed_line2,turning_dict['a19'],turning_dict['a2'],'mixed_1
240 plot_fitted_line(mixed_line3,turning_dict['a3'],turning_dict['a20'],'mixed_1
241 plot_fitted_line(mixed_line4,turning_dict['a21'],turning_dict['a4'],'mixed_1
242 plot_fitted_line(mixed_line5,turning_dict['a5'],turning_dict['a22'],'mixed_1
243 plot_fitted_line(mixed_line6,turning_dict['a23'],turning_dict['a6'],'mixed_1
244
245 # 3. 绘制所有完整圆
246 def plot_full_circle(arc_params, color, label, name):
247     h, k, r = arc_params
248     theta = np.linspace(0, 2 * np.pi, 100)
249     x = h + r * np.cos(theta)
250     y = k + r * np.sin(theta)
251     plt.plot(x, y, color=color, linestyle='--', linewidth=3,
252             label=label)
253     plt.scatter(h, k, color='salmon', s=20)
254     plt.text(h, k+0.12, name, fontsize=20, ha='center')
255
256 # 绘制所有弧线
257 plot_full_circle(arc1, 'pink', 'arc1','04')
258 plot_full_circle(arc2, 'pink', 'arc2','05')
259 plot_full_circle(arc3, 'pink', 'arc3','06')
260 plot_full_circle(arc4, 'pink', 'arc4','07')
261
262 plot_full_circle(mixed_arc1, 'salmon', 'mixed_arc1','01')
263 plot_full_circle(mixed_arc2, 'salmon', 'mixed_arc2','02')
264 plot_full_circle(mixed_arc3, 'salmon', 'mixed_arc3','03')
265
266 # 4. 绘制所有转折点
267 for i in range(1, 24):
268     if f'a{i}' in turning_dict:
269         plt.scatter(turning_dict[f'a{i}'],

```

```

    turning_dict[f'b{i}'], color='black', s=20, zorder=10)
269     if i in [18,20,22]: # 左侧点向左偏移
270         plt.text(turning_dict[f'a{i}']-0.1,
                    turning_dict[f'b{i}'],
271                     f'a{i}', fontsize=20, ha='right')
272     elif i in [19,21,23]: # 右侧点向右偏移
273         plt.text(turning_dict[f'a{i}']+0.1,
                    turning_dict[f'b{i}'],
274                     f'a{i}', fontsize=20, ha='left')
275     else: # 其他点保持原样
276         plt.text(turning_dict[f'a{i}'],
                    turning_dict[f'b{i}'] + 0.05,
277                     f'a{i}', fontsize=20, ha='center')
278
279
280     # 5. 添加图例和标签
281     plt.legend(fontsize=15, loc='upper right')
282     plt.title('工件1_level_fit', fontsize=24)
283     plt.xlabel('x坐标', fontsize=20)
284     plt.ylabel('z坐标', fontsize=20)
285     plt.grid(True, linestyle='--', alpha=0.7)
286     plt.tight_layout()
287
288     # 6. 保存和显示图像
289     plt.savefig('Figures/工件1_level_fit.png', dpi=300,
                bbox_inches='tight')
290     plt.show()
291
292
293 if __name__ == '__main__':
294     #读取数据
295     df=pd.read_excel('第2周C题：接触式轮廓仪的自动标注(2020年D题)/附件1_工件1的测
296     #标记转折点
297     turning_points=['(49.7154,-1.7739)', '(52.7223,-1.7699)', '(57.5009,-1.7676)',
298                     '(64.5999,-1.7690)', '(66.8444,-1.7679)', '(71.6264,-1.7676)',
299                     '(81.8819,-1.7691)', '(84.6165,-1.7795)', '(86.6494,-1.7737)',
300                     '(89.5424,-1.7685)', '(94.3925,-1.7840)', '(101.7909,-1.7828)',
301                     '(106.5504,-1.7766)', '(114.1814,-1.7795)'
302                 ]
303     turning_points_x=[]

```

```

304     turning_points_z=[]
305     for item in turning_points:
306         coord=item.strip('()').split(',')
307         turning_points_x.append(float(coord[0]))
308         turning_points_z.append(float(coord[1]))
309     #转折点命名
310     turning_dict = {}
311     for i in range(len(turning_points)):
312         turning_dict[f'a{i+1}'] = turning_points_x[i]
313         turning_dict[f'b{i+1}'] = turning_points_z[i]
314
315     #绘图
316     plt.figure(figsize=(24,10))
317     plt.plot(df['x'],df['z'],color='gray',linewidth=5)
318     #添加转折点
319     plt.scatter(turning_points_x, turning_points_z, color='gray')
320     for i in range(len(turning_points)):
321         plt.text(turning_points_x[i],turning_points_z[i]+0.08,f'a{i+1}',fontsize
322     plt.title('工件1的测量数据')
323     plt.xlabel('x')
324     plt.ylabel('z')
325     plt.grid(True, linestyle='--', alpha=0.7)
326     plt.tight_layout()
327     plt.savefig('Figures/工件1_level.png')
328     plt.show()
329     plt.close()
330
331     #直线区段拟合
332     print("区间[a1]的拟合结果")
333     line1=linear_fit(df['x'][0],turning_dict['a1'],df)
334     print('区间[a2,a3]的拟合结果: ')
335     line2=linear_fit(turning_dict['a2'],turning_dict['a3'],df)
336     print('区间[a4,a5]的拟合结果: ')
337     line3=linear_fit(turning_dict['a4'],turning_dict['a5'],df)
338     print('区间[a6,a7]的拟合结果: ')
339     line4=linear_fit(turning_dict['a6'],turning_dict['a7'],df)
340     print('区间[a7,a8]的拟合结果: ')
341     line5=linear_fit(turning_dict['a7'],turning_dict['a8'],df)
342     print('区间[a8,a9]的拟合结果: ')
343     line6=linear_fit(turning_dict['a8'],turning_dict['a9'],df)

```

```

344 print('区间[a9,a10]的拟合结果: ')
345 line7=linear_fit(turning_dict['a9'],turning_dict['a10'],df)
346 print('区间[a11,a12]的拟合结果: ')
347 line8=linear_fit(turning_dict['a11'],turning_dict['a12'],df)
348 print('区间[a13,a14]的拟合结果: ')
349 line9=linear_fit(turning_dict['a13'],turning_dict['a14'],df)
350 print('区间[a15,a16]的拟合结果: ')
351 line10=linear_fit(turning_dict['a15'],turning_dict['a16'],df)
352 print("区间[a17,]的拟合结果: ")
353 line11=linear_fit(turning_dict['a17'],df['x'].iloc[-1],df)
354
355 #更新转折点坐标
356 point=line_intersection(line4, line5)
357 turning_dict['a7']=point[0]
358 turning_dict['b7']=point[1]
359 point=line_intersection(line5, line6)
360 turning_dict['a8']=point[0]
361 turning_dict['b8']=point[1]
362 point = line_intersection(line6, line7)
363 turning_dict['a9'] = point[0]
364 turning_dict['b9'] = point[1]
365 #计算z1
366 z1=turning_dict['b8']-turning_dict['b9']
367 print(f'z1={z1:.8f}\n')
368
369 #弧线区段拟合
370 print('区间[a10,a11]的拟合结果: ')
371 arc1=arc_fit(turning_dict['a10'],turning_dict['a11'],df)
372 print('区间[a12,a13]的拟合结果: ')
373 arc2=arc_fit(turning_dict['a12'],turning_dict['a13'],df)
374 print('区间[a14,a15]的拟合结果: ')
375 arc3=arc_fit(turning_dict['a14'],turning_dict['a15'],df)
376 print('区间[a16,a17]的拟合结果: ')
377 arc4=arc_fit(turning_dict['a16'],turning_dict['a17'],df)
378
379 #混合区段拟合
380 print('区间[a1,a2]的拟合结果: ')
381 mixed_line1, mixed_line2,
    mixed_arc1=mixed_fit(turning_dict['a1'],turning_dict['a2'],-1.9,-3.8,-4.3)
382 print('区间[a3,a4]的拟合结果: ')

```

```

383 mixed_line3, mixed_line4,
        mixed_arc2=mixed_fit(turning_dict['a3'],turning_dict['a4'],-2.1,-4.2,-4.3
384 print('区间[a5,a6]的拟合结果: ')
385 mixed_line5, mixed_line6,
        mixed_arc3=mixed_fit(turning_dict['a5'],turning_dict['a6'],-2.1,-4.0,-4.2
386
387 # 更新转折点坐标
388 point=line_intersection(line1,mixed_line1 )
389 turning_dict['a1']=point[0]
390 turning_dict['b1']=point[1]
391 point = line_intersection(line2, mixed_line2)
392 turning_dict['a2'] = point[0]
393 turning_dict['b2'] = point[1]
394 point=line_intersection(line2,mixed_line3)
395 turning_dict['a3']=point[0]
396 turning_dict['b3']=point[1]
397 point=line_intersection(mixed_line4,line3)
398 turning_dict['a4']=point[0]
399 turning_dict['b4']=point[1]
400 point = line_intersection(mixed_line5, line3)
401 turning_dict['a5'] = point[0]
402 turning_dict['b5'] = point[1]
403 point = line_intersection(mixed_line6, line4)
404 turning_dict['a6'] = point[0]
405 turning_dict['b6'] = point[1]
406 point = line_circle_intersection(line7, arc1)[0]
407 turning_dict['a10'] = point[0]
408 turning_dict['b10'] = point[1]
409 point = line_circle_intersection(line8, arc1)[1]
410 turning_dict['a11'] = point[0]
411 turning_dict['b11'] = point[1]
412 point = line_circle_intersection(line8, arc2)[0]
413 turning_dict['a12'] = point[0]
414 turning_dict['b12'] = point[1]
415 point = line_circle_intersection(line9, arc2)[1]
416 turning_dict['a13'] = point[0]
417 turning_dict['b13'] = point[1]
418 point = line_circle_intersection(line9, arc3)[0]
419 turning_dict['a14'] = point[0]
420 turning_dict['b14'] = point[1]

```

```

421 point = line_circle_intersection(line10, arc3)[1]
422 turning_dict['a15'] = point[0]
423 turning_dict['b15'] = point[1]
424 point = line_circle_intersection(line10, arc4)[0]
425 turning_dict['a16'] = point[0]
426 turning_dict['b16'] = point[1]
427 point = line_circle_intersection(line11, arc4)[1]
428 turning_dict['a17'] = point[0]
429 turning_dict['b17'] = point[1]
430 point=line_circle_intersection(mixed_line1,mixed_arc1)
431 turning_dict['a18']=point[0]
432 turning_dict['b18']=point[1]
433 point = line_circle_intersection(mixed_line2, mixed_arc1)
434 turning_dict['a19'] = point[0]
435 turning_dict['b19'] = point[1]
436 point = line_circle_intersection(mixed_line3, mixed_arc2)
437 turning_dict['a20'] = point[0]
438 turning_dict['b20'] = point[1]
439 point = line_circle_intersection(mixed_line4, mixed_arc2)[0]
440 turning_dict['a21'] = point[0]
441 turning_dict['b21'] = point[1]
442 point = line_circle_intersection(mixed_line5, mixed_arc3)
443 turning_dict['a22'] = point[0]
444 turning_dict['b22'] = point[1]
445 point = line_circle_intersection(mixed_line6, mixed_arc3)[0]
446 turning_dict['a23'] = point[0]
447 turning_dict['b23'] = point[1]
448
449 #打印转折点列表
450 print("转折点列表")
451 for i in range(23):
452     print(f'(a{i+1},b{i+1})=({turning_dict[f'a{i+1}']:.8f},{turning_dict[f'b{i+1}']:.8f})')
453 print("圆参数")
454 for i, arc in enumerate([mixed_arc1, mixed_arc2, mixed_arc3], 1):
455     h, k, r = arc
456     print(f'0{i}=(h:.8f},{k:.8f}) 半径={r:.8f}')
457 for i, arc in enumerate([arc1, arc2, arc3, arc4], 4):
458     h, k, r = arc
459     print(f'0{i}=(h:.8f},{k:.8f}) 半径={r:.8f}')
460

```



```

461 #绘制总图
462 plot_all_results(df)

```

5.3 第二问代码

```

1 from Q1_main import line_intersection, line_circle_intersection,
    linear_fit, arc_fit, mixed_fit
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import os
6 import math
7
8 #设置字体
9 plt.rcParams['font.sans-serif'] = ['SimHei']
10 #设置负号
11 plt.rcParams['axes.unicode_minus'] = False
12
13 os.makedirs('Figures', exist_ok=True)
14
15 def angle_search(*lines):
16     slopes = [line[0] for line in lines]
17     return np.mean([math.atan(s) for s in slopes])
18
19 def coordinate_tranaform(df, angle):
20     df['x_new'] = df['x'] * np.cos(angle) - df['z'] * np.sin(angle)
21     df['z_new'] = df['x'] * np.sin(angle) + df['z'] * np.cos(angle)
22     df['x'] = df['x_new'].round(8)
23     df['z'] = df['z_new'].round(8)
24
25     df.drop(columns=['x_new', 'z_new'], inplace=True)
26     df.to_excel('Figures/工件1_down_new.xlsx', sheet_name='down',
                index=False)
27     return df
28
29 def plot_all_results(df):
30     plt.figure(figsize=(24, 10))
31
32     # 1. 绘制原始数据

```

```

33 plt.plot(df['x'], df['z'], color='gray', linewidth=5,
34          label='原始数据')
35
36 # 2. 绘制所有直线段拟合曲线
37 def plot_fitted_line(line, start, end, line_name,
38                      color='skyblue'):
39     slope, _, intercept = line
40     x_fit = np.linspace(start, end, 100)
41     y_fit = slope * x_fit + intercept
42     plt.plot(x_fit, y_fit, color=color, linestyle='--',
43             linewidth=3, label=line_name)
44
45 plot_fitted_line(line1, df['x'][0], turning_dict['a1'], 'line1')
46 plot_fitted_line(line2, turning_dict['a2'], turning_dict['a3'],
47                  'line2')
48 plot_fitted_line(line3, turning_dict['a4'], turning_dict['a5'],
49                  'line3')
50 plot_fitted_line(line4, turning_dict['a6'], turning_dict['a7'],
51                  'line4')
52 plot_fitted_line(line5, turning_dict['a7'], turning_dict['a8'],
53                  'line5')
54 plot_fitted_line(line6, turning_dict['a8'], turning_dict['a9'],
55                  'line6')
56 plot_fitted_line(line7, turning_dict['a9'], turning_dict['a10'],
57                  'line7')
58 plot_fitted_line(line8, turning_dict['a11'],
59                  turning_dict['a12'], 'line8')
60 plot_fitted_line(line9, turning_dict['a13'],
61                  turning_dict['a14'], 'line9')
62 plot_fitted_line(line10, turning_dict['a15'],
63                  turning_dict['a16'], 'line10')
64 plot_fitted_line(line11, turning_dict['a17'], df['x'].iloc[-1],
65                  'line11')
66
67 plot_fitted_line(mixed_line1, turning_dict['a1'],
68                  turning_dict['a18'], 'mixed_line1', color='blue')
69 plot_fitted_line(mixed_line2, turning_dict['a19'],
70                  turning_dict['a2'], 'mixed_line2', color='blue')
71 plot_fitted_line(mixed_line3, turning_dict['a3'],
72                  turning_dict['a20'], 'mixed_line3', color='blue')

```

```

57 plot_fitted_line(mixed_line4, turning_dict['a21'],
    turning_dict['a4'], 'mixed_line4', color='blue')
58 plot_fitted_line(mixed_line5, turning_dict['a5'],
    turning_dict['a22'], 'mixed_line5', color='blue')
59 plot_fitted_line(mixed_line6, turning_dict['a23'],
    turning_dict['a6'], 'mixed_line6', color='blue')
60
61 # 3. 绘制所有完整圆
62 def plot_full_circle(arc_params, color, label, name):
63     h, k, r = arc_params
64     theta = np.linspace(0, 2 * np.pi, 100)
65     x = h + r * np.cos(theta)
66     y = k + r * np.sin(theta)
67     plt.plot(x, y, color=color, linestyle='--', linewidth=3,
        label=label)
68     plt.scatter(h, k, color='salmon', s=20)
69     plt.text(h, k + 0.12, name, fontsize=20, ha='center')
70
71 # 绘制所有弧线
72 plot_full_circle(arc1, 'pink', 'arc1', '04')
73 plot_full_circle(arc2, 'pink', 'arc2', '05')
74 plot_full_circle(arc3, 'pink', 'arc3', '06')
75 plot_full_circle(arc4, 'pink', 'arc4', '07')
76
77 plot_full_circle(mixed_arc1, 'salmon', 'mixed_arc1', '01')
78 plot_full_circle(mixed_arc2, 'salmon', 'mixed_arc2', '02')
79 plot_full_circle(mixed_arc3, 'salmon', 'mixed_arc3', '03')
80
81 # 4. 绘制所有转折点
82 for i in range(1, 24):
83     if f'a{i}' in turning_dict:
84         plt.scatter(turning_dict[f'a{i}'],
            turning_dict[f'b{i}'], color='black', s=20, zorder=10)
85         if i in [18, 20, 22]: # 左侧点向左偏移
86             plt.text(turning_dict[f'a{i}'] - 0.1,
                turning_dict[f'b{i}'],
87                 f'a{i}', fontsize=20, ha='right')
88         elif i in [19, 21, 23]: # 右侧点向右偏移
89             plt.text(turning_dict[f'a{i}'] + 0.1,
                turning_dict[f'b{i}'],

```

```

90         f'a{i}', fontsize=20, ha='left')
91     else: # 其他点保持原样
92         plt.text(turning_dict[f'a{i}'],
93                 turning_dict[f'b{i}'] + 0.05,
94                 f'a{i}', fontsize=20, ha='center')
95
96 # 5. 添加图例和标签
97 plt.legend(fontsize=15, loc='upper right')
98 plt.title('工件1_down_fit', fontsize=24)
99 plt.xlabel('x坐标', fontsize=20)
100 plt.ylabel('z坐标', fontsize=20)
101 plt.grid(True, linestyle='--', alpha=0.7)
102 plt.tight_layout()
103
104 # 6. 保存和显示图像
105 plt.savefig('Figures/工件1_down_fit.png', dpi=300,
106             bbox_inches='tight')
107 plt.show()
108
109 if __name__ == '__main__':
110     # 读取数据
111     df =
112         pd.read_excel('第2周C题：接触式轮廓仪的自动标注(2020年D题)/附件1_工件1的测
113             sheet_name='down')
114
115     # 标记转折点
116     turning_points = ['(52.2842,0.9468)', '(55.1646,0.5689)',
117                     '(60.0920,-0.0733)', '(62.1717,-0.3424)',
118                     '(67.1081,-0.9914)', '(69.1762,-1.2660)',
119                     '(74.1437,-1.9026)', '(79.2127,-1.5718)',
120                     '(84.1239,-3.2049)', '(87.0161,-3.5982)',
121                     '(88.8237,-3.8364)', '(89.7228,-3.9415)',
122                     '(91.7218,-4.1919)', '(96.5639,-4.8494)',
123                     '(103.7796,-5.7907)', '(108.6285,-6.4093)',
124                     '(116.0250,-7.3731)']
125
126     turning_points_x = []
127     turning_points_z = []
128     for item in turning_points:
129         coord = item.strip('()').split(',')

```

```

122     turning_points_x.append(float(coord[0]))
123     turning_points_z.append(float(coord[1]))
124 # 转折点命名
125 turning_dict = {}
126 for i in range(len(turning_points)):
127     turning_dict[f'a{i + 1}'] = turning_points_x[i]
128     turning_dict[f'b{i + 1}'] = turning_points_z[i]
129
130 # 绘图
131 plt.figure(figsize=(24, 10))
132 plt.plot(df['x'], df['z'], color='gray', linewidth=5)
133 # 添加转折点
134 plt.scatter(turning_points_x, turning_points_z, color='gray')
135 for i in range(len(turning_points)):
136     plt.text(turning_points_x[i], turning_points_z[i] + 0.08,
137              f'a{i + 1}', fontsize=20)
138
139 # 直线区段拟合
140 print("区间[,a1]的拟合结果")
141 line1 = linear_fit(df['x'][0], turning_dict['a1'], df)
142 print('区间[a2,a3]的拟合结果: ')
143 line2 = linear_fit(turning_dict['a2'], turning_dict['a3'], df)
144 print('区间[a4,a5]的拟合结果: ')
145 line3 = linear_fit(turning_dict['a4'], turning_dict['a5'], df)
146 print('区间[a6,a7]的拟合结果: ')
147 line4 = linear_fit(turning_dict['a6'], turning_dict['a7'], df)
148 print('区间[a9,a10]的拟合结果: ')
149 line7 = linear_fit(turning_dict['a9'], turning_dict['a10'], df)
150 print('区间[a11,a12]的拟合结果: ')
151 line8 = linear_fit(turning_dict['a11'], turning_dict['a12'], df)
152 print('区间[a13,a14]的拟合结果: ')
153 line9 = linear_fit(turning_dict['a13'], turning_dict['a14'], df)
154 print('区间[a15,a16]的拟合结果: ')
155 line10 = linear_fit(turning_dict['a15'], turning_dict['a16'], df)
156 print("区间[a17,]的拟合结果: ")
157 line11 = linear_fit(turning_dict['a17'], df['x'].iloc[-1], df)
158
159 # 计算角度
angle = -angle_search(line1, line2, line3, line4, line7, line8,
                      line9, line10, line11)

```

```

160     print(f'angle= {angle}')
```

```

161
```

```

162     # 坐标变换
163     df = coordinate_tranaform(df, angle)
164
```

```

165     # 计算新的转折点坐标
166     turning_points_x_new = []
167     turning_points_z_new = []
168     for i in range(len(turning_points)):
169         turning_points_x_new.append(turning_points_x[i] *
170                                     np.cos(angle) - turning_points_z[i] * np.sin(angle))
171         turning_points_z_new.append(turning_points_x[i] *
172                                     np.sin(angle) + turning_points_z[i] * np.cos(angle))
173     turning_points_x=turning_points_x_new
174     turning_points_z=turning_points_z_new
175
```

```

176     # 转折点更新
177     for i in range(len(turning_points)):
178         turning_dict[f'a{i + 1}'] = turning_points_x[i]
179         turning_dict[f'b{i + 1}'] = turning_points_z[i]
180
```

```

181     # 绘图
182     plt.plot(df['x'], df['z'], color='black', linewidth=3)
183     # 添加转折点
184     plt.scatter(turning_points_x, turning_points_z, color='black')
185     for i in range(len(turning_points)):
186         plt.text(turning_points_x[i], turning_points_z[i] + 0.08,
187                 f'a{i + 1}', fontsize=15)
188     plt.title('工件1_down')
189     plt.xlabel('x')
190     plt.ylabel('z')
191     plt.grid(True, linestyle='--', alpha=0.7)
192     plt.tight_layout()
193     plt.savefig('Figures/工件1_down.png')
194     plt.show()
195     plt.close()
196
```

```

197     # 直线区段拟合
198     print("区间[,a1]的拟合结果")
199     line1 = linear_fit(df['x'][0], turning_dict['a1'], df)

```

```

197 print('区间[a2,a3]的拟合结果: ')
198 line2 = linear_fit(turning_dict['a2'], turning_dict['a3'], df)
199 print('区间[a4,a5]的拟合结果: ')
200 line3 = linear_fit(turning_dict['a4'], turning_dict['a5'], df)
201 print('区间[a6,a7]的拟合结果: ')
202 line4 = linear_fit(turning_dict['a6'], turning_dict['a7'], df)
203 print('区间[a7,a8]的拟合结果: ')
204 line5 = linear_fit(turning_dict['a7'], turning_dict['a8'], df)
205 print('区间[a8,a9]的拟合结果: ')
206 line6 = linear_fit(turning_dict['a8'], turning_dict['a9'], df)
207 print('区间[a9,a10]的拟合结果: ')
208 line7 = linear_fit(turning_dict['a9'], turning_dict['a10'], df)
209 print('区间[a11,a12]的拟合结果: ')
210 line8 = linear_fit(turning_dict['a11'], turning_dict['a12'], df)
211 print('区间[a13,a14]的拟合结果: ')
212 line9 = linear_fit(turning_dict['a13'], turning_dict['a14'], df)
213 print('区间[a15,a16]的拟合结果: ')
214 line10 = linear_fit(turning_dict['a15'], turning_dict['a16'], df)
215 print("区间[a17,]的拟合结果: ")
216 line11 = linear_fit(turning_dict['a17'], df['x'].iloc[-1], df)
217
218 # 更新转折点坐标
219 point = line_intersection(line4, line5)
220 turning_dict['a7'] = point[0]
221 turning_dict['b7'] = point[1]
222 point = line_intersection(line5, line6)
223 turning_dict['a8'] = point[0]
224 turning_dict['b8'] = point[1]
225 point = line_intersection(line6, line7)
226 turning_dict['a9'] = point[0]
227 turning_dict['b9'] = point[1]
228
229 # 计算z1
230 z1 = turning_dict['b8'] - turning_dict['b9']
231 print(f'z1={z1:.8f}\n')
232
233 # 弧线区段拟合
234 print('区间[a10,a11]的拟合结果: ')
235 arc1 = arc_fit(turning_dict['a10'], turning_dict['a11'], df)
236 print('区间[a12,a13]的拟合结果: ')

```

```

237 arc2 = arc_fit(turning_dict['a12'], turning_dict['a13'],df)
238 print('区间[a14,a15]的拟合结果: ')
239 arc3 = arc_fit(turning_dict['a14'], turning_dict['a15'],df)
240 print('区间[a16,a17]的拟合结果: ')
241 arc4 = arc_fit(turning_dict['a16'], turning_dict['a17'],df)
242
243 # 混合区段拟合
244 print('区间[a1,a2]的拟合结果: ')
245 mixed_line1, mixed_line2, mixed_arc1 =
    mixed_fit(turning_dict['a1'], turning_dict['a2'], 7.5, 5.3,
    5.2,df)
246 print('区间[a3,a4]的拟合结果: ')
247 mixed_line3, mixed_line4, mixed_arc2 =
    mixed_fit(turning_dict['a3'], turning_dict['a4'], 7.7, 5.3,
    5.2,df)
248 print('区间[a5,a6]的拟合结果: ')
249 mixed_line5, mixed_line6, mixed_arc3 =
    mixed_fit(turning_dict['a5'], turning_dict['a6'], 7.7, 5.3,
    5.3,df)
250
251 # 更新转折点坐标
252 point = line_intersection(line1, mixed_line1)
253 turning_dict['a1'] = point[0]
254 turning_dict['b1'] = point[1]
255 point = line_intersection(line2, mixed_line2)
256 turning_dict['a2'] = point[0]
257 turning_dict['b2'] = point[1]
258 point = line_intersection(line2, mixed_line3)
259 turning_dict['a3'] = point[0]
260 turning_dict['b3'] = point[1]
261 point = line_intersection(mixed_line4, line3)
262 turning_dict['a4'] = point[0]
263 turning_dict['b4'] = point[1]
264 point = line_intersection(mixed_line5, line3)
265 turning_dict['a5'] = point[0]
266 turning_dict['b5'] = point[1]
267 point = line_intersection(mixed_line6, line4)
268 turning_dict['a6'] = point[0]
269 turning_dict['b6'] = point[1]
270 point = line_circle_intersection(line7, arc1)[0]

```



```

271     turning_dict['a10'] = point[0]
272     turning_dict['b10'] = point[1]
273     point = line_circle_intersection(line8, arc1)[1]
274     turning_dict['a11'] = point[0]
275     turning_dict['b11'] = point[1]
276     point = line_circle_intersection(line8, arc2)[0]
277     turning_dict['a12'] = point[0]
278     turning_dict['b12'] = point[1]
279     point = line_circle_intersection(line9, arc2)[1]
280     turning_dict['a13'] = point[0]
281     turning_dict['b13'] = point[1]
282     point = line_circle_intersection(line9, arc3)[0]
283     turning_dict['a14'] = point[0]
284     turning_dict['b14'] = point[1]
285     point = line_circle_intersection(line10, arc3)[1]
286     turning_dict['a15'] = point[0]
287     turning_dict['b15'] = point[1]
288     point = line_circle_intersection(line10, arc4)[0]
289     turning_dict['a16'] = point[0]
290     turning_dict['b16'] = point[1]
291     point = line_circle_intersection(line11, arc4)[1]
292     turning_dict['a17'] = point[0]
293     turning_dict['b17'] = point[1]
294     point = line_circle_intersection(mixed_line1, mixed_arc1)
295     turning_dict['a18'] = point[0]
296     turning_dict['b18'] = point[1]
297     point = line_circle_intersection(mixed_line2, mixed_arc1)
298     turning_dict['a19'] = point[0]
299     turning_dict['b19'] = point[1]
300     point = line_circle_intersection(mixed_line3, mixed_arc2)
301     turning_dict['a20'] = point[0]
302     turning_dict['b20'] = point[1]
303     point = line_circle_intersection(mixed_line4, mixed_arc2)
304     turning_dict['a21'] = point[0]
305     turning_dict['b21'] = point[1]
306     point = line_circle_intersection(mixed_line5, mixed_arc3)
307     turning_dict['a22'] = point[0]
308     turning_dict['b22'] = point[1]
309     point = line_circle_intersection(mixed_line6, mixed_arc3)
310     turning_dict['a23'] = point[0]

```

```

311     turning_dict['b23'] = point[1]
312
313     # 打印转折点列表
314     print("转折点列表")
315     for i in range(23):
316         print(f'(a{i + 1},b{i + 1})=({turning_dict[f'a{i + 1}']:.8f},{turning_dict[f'b{i + 1}']:.8f})')
317     print("圆参数")
318     for i, arc in enumerate([mixed_arc1, mixed_arc2, mixed_arc3], 1):
319         h, k, r = arc
320         print(f'0{i}={({h:.8f},{k:.8f}) 半径={r:.8f}')
321     for i, arc in enumerate([arc1, arc2, arc3, arc4], 4):
322         h, k, r = arc
323         print(f'0{i}={({h:.8f},{k:.8f}) 半径={r:.8f}')
324
325     # 绘制总图
326     plot_all_results(df)

```

5.4 第三问代码

```

1 from Q1_main import linear_fit,line_intersection
2 from Q2_main import coordinate_transform
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import os
7 import math
8
9 #设置字体
10 plt.rcParams['font.sans-serif'] = ['SimHei']
11 #设置负号
12 plt.rcParams['axes.unicode_minus'] = False
13
14 os.makedirs('Figures', exist_ok=True)
15
16 if __name__ == '__main__':
17     # 读取数据
18     # 创建ExcelFile对象
19     xls =

```

```

    pd.ExcelFile('第2周C题：接触式轮廓仪的自动标注(2020年D题)/附件2_工件2的整
20
21 # 读取所有sheet到字典
22 sheets = {sheet: xls.parse(sheet) for sheet in
    xls.sheet_names[:10]}
23
24 plt.figure(figsize=(12, 8))
25
26 # 遍历所有sheet
27 for sheet_name, df in sheets.items():
28     # 绘制x和z列
29     plt.plot(df.iloc[:, 0], df.iloc[:, 1], label=sheet_name)
30
31 # 添加图例和标签
32 plt.legend(loc='best')
33 plt.xlabel('X')
34 plt.ylabel('Z')
35 plt.title('Q3 All Sheets Data')
36 plt.grid(True, linestyle='--', alpha=0.7)
37 plt.tight_layout()
38 plt.savefig('Figures/Q3_All_Sheets_Data.png')
39 plt.show()
40
41 # 标记5组区间
42 interval_axis={
43     'p25': (58, 59.6),
44     'p26': (56.8, 58.6),
45     'p27': (54.6, 56.2),
46     'p28': (52.4, 54),
47     'p29': (55.5, 56.8)
48 }
49 interval_axis_vertical={
50     'p25': (60.5, 61.2),
51     'p26': (59.2, 60),
52     'p27': (57, 57.8),
53     'p28': (54.8, 55.8),
54     'p29': (57.4, 58.4)
55 }
56 # 按顺序处理每两个sheet对应一个区间
57 lines={}

```

```

58     lines_vertical={}
59     intersection_points={}
60     for i, (key, (start, end)) in enumerate(interval_axis.items()):
61         sheet1_idx = i * 2
62         sheet2_idx = sheet1_idx + 1
63
64         # 确保索引不越界
65         if sheet2_idx >= len(sheets):
66             break
67
68         sheet1_name = list(sheets.keys())[sheet1_idx]
69         sheet2_name = list(sheets.keys())[sheet2_idx]
70
71         # 获取两个sheet的数据
72         df1 = sheets[sheet1_name]
73         df2 = sheets[sheet2_name]
74
75         print(f"区间 {key}({start}-{end}) 对应的sheet: {sheet1_name}
76               和 {sheet2_name}")
77
78         # 线性拟合
79         line1=linear_fit(start,end,df1)
80         line2=linear_fit(start,end,df2)
81         lines[sheet1_name] = line1
82         lines[sheet2_name] = line2
83
84         line1_vertical=linear_fit(interval_axis_vertical[key][0],interval_axis_v
85         line2_vertical=linear_fit(interval_axis_vertical[key][0],interval_axis_v
86         lines_vertical[sheet1_name]=line1_vertical
87         lines_vertical[sheet2_name]=line2_vertical
88
89         # 计算交点
90         point1=line_intersection(line1,line1_vertical)
91         point2=line_intersection(line2,line2_vertical)
92
93         intersection_points[sheet1_name]=point1
94         intersection_points[sheet2_name]=point2
95
96         # print(lines)
97
98         # 计算倾角

```

```

97 slopes = [line[0] for line in lines.values()]
98 angles={}
99 angles_deg={}
100 for i in range(len(slopes)):
101     angle=math.atan(slopes[i])
102     key_name=list(lines.keys())
103     angles[key_name[i]]=angle
104     angles_deg[key_name[i]]=angle*180/math.pi
105 alpha_ref=np.mean(list(angles.values()))
106 alphas_i={}
107 for i in range(len(angles)):
108     alphas_i[key_name[i]]=angles[key_name[i]]-alpha_ref
109
110 print(f"角度(弧度):{angles}")
111 print(f"角度(角度):{angles_deg}")
112 print(f"标准角度(弧度):{alpha_ref}, 标准角度(角度):{alpha_ref *
113     180 / math.pi}")
114 print(f"相对角度(弧度):{alphas_i}")
115 print(f"相对角度(角度):{np.array(list(alphas_i.values()))*180/math.pi}\n")
116
117 # 转换交点坐标
118 for key, point in intersection_points.items():
119     # 获取当前角度差
120     alpha = alphas_i[key]
121     # 旋转变换
122     x = point[0] * math.cos(alpha) - point[1] * math.sin(alpha)
123     y = point[0] * math.sin(alpha) + point[1] * math.cos(alpha)
124     intersection_points[key] = (x, y)
125
126 #计算转换后的质心
127 centroid_x = np.mean([point[0] for point in
128     intersection_points.values()])
129 centroid_z = np.mean([point[1] for point in
130     intersection_points.values()])
131 print(f"交点的质心: ({centroid_x}, {centroid_z})")
132
133 # 计算每个点的x轴平移量
134 translations_x = {key: centroid_x - point[0] for key, point in
135     intersection_points.items()}
136 print("每个的x轴平移量:",translations_x)

```

```

133
134 # 转换坐标
135 data_x={}
136 data_z={}
137 os.makedirs('Figures/Q3_excels', exist_ok=True)
138 alpha_values = list(alphas_i.values())
139 plt.figure(figsize=(12, 8))
140 for i, (sheet_name, df) in enumerate(sheets.items()):
141     df = coordinate_transform(df, alpha_values[i],
142                               save_path=f'Figures/Q3_excels/Q3_{sheet_name}_new.xlsx')
143     # 平移
144     df.iloc[:, 0] += translations_x[sheet_name]
145     # 保存x数据
146     data_x[sheet_name]=df.iloc[:, 0].values
147     # 保存z数据
148     data_z[sheet_name]=df.iloc[:, 1].values
149     # 绘图
150     plt.plot(df.iloc[:, 0], df.iloc[:, 1], label=sheet_name)
151 plt.legend(loc='best')
152 plt.xlabel('X')
153 plt.ylabel('Z')
154 plt.title('Q3_Transformed')
155 plt.grid(True, linestyle='--', alpha=0.7)
156 plt.tight_layout()
157 plt.savefig('Figures/Q3_All_Sheets_Data_Transformed.png')
158 plt.show()
159
160 ##平均并绘图
161 # 1. 确定全局x轴范围（所有工作表的x值合并后的范围）
162 all_x_values = []
163 for x_array in data_x.values():
164     all_x_values.extend(x_array)
165
166 global_x_min = np.min(all_x_values)
167 global_x_max = np.max(all_x_values)
168 print(f"全局x轴范围: [{global_x_min:.4f}, {global_x_max:.4f}]")
169
170 # 2. 创建参考x轴（在全局范围内均匀采样）
171 num_samples = 20000
172 reference_x = np.linspace(global_x_min, global_x_max,

```

```

num_samples)
172
173 # 3. 为每个参考x点匹配所有工作表的z值并计算平均
174 average_z = []
175
176 for ref_x in reference_x:
177     z_matches = []
178
179     # 遍历每个工作表，找到最接近参考x的z值
180     for sheet in data_x.keys():
181         # 获取当前工作表的x和z数据
182         x_vals = data_x[sheet]
183         z_vals = data_z[sheet]
184
185         # 找到最接近参考x的索引
186         closest_idx = np.argmin(np.abs(x_vals - ref_x))
187         # 收集对应的z值
188         z_matches.append(z_vals[closest_idx])
189
190     # 计算当前参考x点的平均z值
191     avg_z = np.mean(z_matches)
192     average_z.append(avg_z)
193
194 # 4. 生成结果数据框
195 result_df = pd.DataFrame({
196     'x': reference_x,
197     'z': average_z
198 })
199
200 # 5. 保存结果
201 result_df.to_excel('Figures/Q3_Avg_Z_Results.xlsx', index=False)
202 print(f"结果已保存到 Figures/Q3_Avg_Z_Results.xlsx")
203
204 # 6. 绘制平均曲线
205 plt.figure(figsize=(12, 8))
206 plt.plot(result_df['x'], result_df['z'], color='blue',
207          linewidth=2.5, label='平均Z值')
208
209 # 可选：叠加显示原始数据曲线（浅灰色）作为参考
210 for sheet in data_x.keys():

```

```

210         plt.plot(data_x[sheet], data_z[sheet], color='lightgray',
211                  alpha=0.5)
212
213     plt.xlabel('X坐标', fontsize=12)
214     plt.ylabel('平均Z坐标', fontsize=12)
215     plt.title('全局X范围内的平均Z值分布', fontsize=14)
216     plt.grid(True, linestyle='--', alpha=0.7)
217     plt.legend()
218     plt.tight_layout()
219     plt.savefig('Figures/Q3_Avg_Z_Results.png', dpi=300)
220     plt.show()
221
222 import numpy as np
223 import pandas as pd
224 import matplotlib.pyplot as plt
225 import os
226 import math
227 from Q1_main import
228     data_reader, angle_cal, line_intersection, line_circle_intersection, linear_fit, a
229
230 plt.rcParams['font.sans-serif'] = ['SimHei']
231 plt.rcParams['axes.unicode_minus'] = False
232 os.makedirs('Figures', exist_ok=True)
233
234 # 区间
235 # [45.155, 61.25]
236 interval = [45.8, 47.4, 55.1, 57.5, 58.9, 59.5, 59.8, 60.3, 60.65, 61.25]
237
238 # 读取数据
239 df = pd.read_excel("Figures/Q3_Avg_Z_Results.xlsx")
240
241 # 转折点命名
242 turning_dict = {}
243 # 为每个区间点找到最近的x对应的z值
244 for i, x_val in enumerate(interval):
245     # 计算每个x与目标值的绝对差
246     differences = np.abs(df['x'] - x_val)
247
248     # 找到最小差值的索引

```



```

248     min_index = differences.idxmin()
249
250     # 获取对应的z值
251     corresponding_z = df.loc[min_index, 'z']
252
253     # 存储到字典中, a为x值, b为对应的z值
254     turning_dict[f'a{i + 1}'] = df.loc[min_index, 'x']
255     turning_dict[f'b{i + 1}'] = corresponding_z
256
257 # 绘图
258 plt.figure(figsize=(18, 6))
259 plt.plot(df['x'], df['z'], color='gray', linewidth=3)
260 # 添加转折点
261 i=0
262 for i in range(len(interval)):
263     plt.scatter(turning_dict[f'a{i + 1}'], turning_dict[f'b{i +
264         1}'], s=30, color='black',zorder=10)
265     plt.text(turning_dict[f'a{i + 1}'], turning_dict[f'b{i + 1}'] +
266         0.08, f'a{i + 1}', fontsize=20)
267     i+=1
268 plt.title('工件2的测量数据')
269 plt.xlabel('x')
270 plt.ylabel('z')
271 plt.grid(True, linestyle='--', alpha=0.7)
272 plt.tight_layout()
273 plt.savefig('Figures/Q3_data.png')
274 plt.show()
275 plt.close()
276
277 # 片段拟合
278 print('区间[a1,a2]的拟合结果: ')
279 line1=linear_fit(turning_dict['a1'],turning_dict['a2'],df,
280     epsilon=0.1)
281 print('区间[a2,a3]的拟合结果: ')
282 arc1=arc_fit(turning_dict['a2'],turning_dict['a3'],df,epsilon=0.2)
283 print('区间[a3,a4]的拟合结果: ')
284 line2=linear_fit(turning_dict['a3'],turning_dict['a4'],df,epsilon=0.2)
285 print('区间[a4,a5]的拟合结果: ')
286 line3=linear_fit(turning_dict['a4'],turning_dict['a5'],df,epsilon=0.1)
287 print('区间[a5,a6]的拟合结果: ')

```

```

285 line4=linear_fit(turning_dict['a5'],turning_dict['a6'],df,epsilon=0.01)
286 print('区间[a6,a7]的拟合结果: ')
287 line5=linear_fit(turning_dict['a6'],turning_dict['a7'],df,epsilon=0.01)
288 print('区间[a7,a8]的拟合结果: ')
289 line6=linear_fit(turning_dict['a7'],turning_dict['a8'],df,epsilon=0.01)
290 print('区间[a8,a9]的拟合结果: ')
291 line7=linear_fit(turning_dict['a8'],turning_dict['a9'],df,epsilon=0.01)
292 print('区间[a9,a10]的拟合结果: ')
293 line8=linear_fit(turning_dict['a9'],turning_dict['a10'],df,epsilon=0.01)
294
295 #更新转折点
296 point=line_circle_intersection(line1,arc1)[0]
297 turning_dict['a2']=point[0]
298 turning_dict['b2']=point[1]
299 point=line_circle_intersection(line2,arc1)[1]
300 turning_dict['a3']=point[0]
301 turning_dict['b3']=point[1]
302 point=line_intersection(line2,line3)
303 turning_dict['a4']=point[0]
304 turning_dict['b4']=point[1]
305 point=line_intersection(line3,line4)
306 turning_dict['a5']=point[0]
307 turning_dict['b5']=point[1]
308 point=line_intersection(line4,line5)
309 turning_dict['a6']=point[0]
310 turning_dict['b6']=point[1]
311 point=line_intersection(line5,line6)
312 turning_dict['a7']=point[0]
313 turning_dict['b7']=point[1]
314 point=line_intersection(line6,line7)
315 turning_dict['a8']=point[0]
316 turning_dict['b8']=point[1]
317 point=line_intersection(line7,line8)
318 turning_dict['a9']=point[0]
319 turning_dict['b9']=point[1]
320
321 # 打印转折点列表
322 print("转折点列表")
323 for i in range(10):
324     print(f'(a{i + 1},b{i + 1})=({turning_dict[f'a{i + 1}']},

```

```

1}']:.8f},{turning_dict[f'b{i + 1}']:.8f}))')
325 print("圆参数")
326 h, k, r = arc1
327 print(f'01=({h:.8f},{k:.8f}) 半径={r:.8f}')
328
329 # 绘制总图
330 def plot_all_results(df):
331     plt.figure(figsize=(18, 8))
332
333     # 1. 绘制原始数据
334     plt.plot(df['x'], df['z'], color='gray', linewidth=4,
335             label='原始数据')
336
337     # 2. 绘制所有直线段拟合曲线
338     def plot_fitted_line(line, start, end, line_name,
339                         color='skyblue'):
340         slope, _, intercept = line
341         x_fit = np.linspace(start, end, 100)
342         y_fit = slope * x_fit + intercept
343         plt.plot(x_fit, y_fit, color=color, linestyle='--',
344                 linewidth=3, label=line_name)
345
346     plot_fitted_line(line1, turning_dict['a1'], turning_dict['a2'],
347                     'line1')
348     plot_fitted_line(line2, turning_dict['a3'], turning_dict['a4'],
349                     'line2')
350     plot_fitted_line(line3, turning_dict['a4'], turning_dict['a5'],
351                     'line3')
352     plot_fitted_line(line4, turning_dict['a5'], turning_dict['a6'],
353                     'line4')
354     plot_fitted_line(line5, turning_dict['a6'], turning_dict['a7'],
355                     'line5')
356     plot_fitted_line(line6, turning_dict['a7'], turning_dict['a8'],
357                     'line6')
358     plot_fitted_line(line7, turning_dict['a8'], turning_dict['a9'],
359                     'line7')
360     plot_fitted_line(line8, turning_dict['a9'], turning_dict['a10'],
361                     'line8')
362
363     # 3. 绘制所有完整圆

```

```

353 def plot_full_circle(arc_params, color, label, name):
354     h, k, r = arc_params
355     theta = np.linspace(0, 2 * np.pi, 100)
356     x = h + r * np.cos(theta)
357     y = k + r * np.sin(theta)
358     plt.plot(x, y, color=color, linestyle='--', linewidth=3,
359             label=label)
360     plt.scatter(h, k, color='salmon', s=20)
361     plt.text(h, k + 0.12, name, fontsize=20, ha='center')
362
363 # 绘制所有弧线
364 plot_full_circle(arc1, 'pink', 'arc1', '01')
365
366 # 4. 绘制所有转折点
367 for i in range(1, 11):
368     if f'a{i}' in turning_dict:
369         plt.scatter(turning_dict[f'a{i}'],
370                     turning_dict[f'b{i}'], color='black', s=30, zorder=10)
371         plt.text(turning_dict[f'a{i}'], turning_dict[f'b{i}'],
372                 f'a{i}', fontsize=24, ha='center')
373
374 # 5. 添加图例和标签
375 plt.legend(fontsize=20, loc='upper right')
376 plt.title('Q3_data_fit', fontsize=24)
377 plt.xlabel('x坐标', fontsize=20)
378 plt.ylabel('z坐标', fontsize=20)
379 plt.grid(True, linestyle='--', alpha=0.7)
380 plt.tight_layout()
381
382 # 6. 保存和显示图像
383 plt.savefig('Figures/Q3_data_fit.png', dpi=300,
384             bbox_inches='tight')
385 plt.show()
386
387 plot_all_results(df)
388
389 # 计算各拟合片段之间的长度
390 print("轮廓片段之间长度：")
391 line_length={}
392 for i in range(1,10):

```

```

389     line_length[f'line{i}']=np.sqrt((turning_dict[f'a{i+1}']-turning_dict[f'a{i}']
390     print(f'(a{i },a{i + 1})={line_length[f'line{i}']:.8f}')
391
392 # 获取圆弧参数
393 h, k, r = arc1
394
395 # 获取转折点坐标
396 a2_x, a2_z = turning_dict['a2'], turning_dict['b2']
397 a3_x, a3_z = turning_dict['a3'], turning_dict['b3']
398
399 # 计算两点相对于圆心的角度
400 theta1 = math.atan2(a2_z - k, a2_x - h)
401 theta2 = math.atan2(a3_z - k, a3_x - h)
402
403 # 计算圆弧角度（确保角度为正）
404 delta_theta = abs(theta2 - theta1)
405 if delta_theta > math.pi:
406     delta_theta = 2 * math.pi - delta_theta
407
408 # 计算圆弧长度
409 arc_length = r * delta_theta
410 print(f'圆弧长度(a2,a3)={arc_length:.8f}')

```